

ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ
Τμήμα Ψηφιακών Συστημάτων
Μάθημα: «Δικτυοκεντρικά Πληροφοριακά Συστήματα»
Ακαδημαϊκό έτος 2024-2025

Ονοματεπώνυμο: Παναγιώτης Χονδρομπίλας

ΑΜ: Ε20190

Αναφορά Απαλλακτικής Εργασίας

Κεφάλαιο 1: Εισαγωγή ……2

Κεφάλαιο 2: Περιγραφή συστήματος που υλοποιήθηκε ……..3

Κεφάλαιο 3: Υλοποίηση συστήματος ……5

Κεφάλαιο 4: Εγχειρίδιο διαχειριστή ……..46

Κεφάλαιο 5: Εγχειρίδιο χρήστη ……48

Κεφάλαιο 1: Εισαγωγή

Η παρούσα εργασία αφορά την ανάπτυξη ενός δικτυοκεντρικού πληροφοριακού συστήματος για τη διαχείριση χρηστών και κρατήσεων ενός γυμναστηρίου. Το σύστημα έχει σχεδιαστεί ώστε να καλύπτει τις ανάγκες ενός σύγχρονου γυμναστηρίου, προσφέροντας εργαλεία διαχείρισης τόσο στους διαχειριστές όσο και στους χρήστες του. Το πληροφοριακό σύστημα αποτελείται από δύο κύρια υποσυστήματα: το διαχειριστικό υποσύστημα και το υποσύστημα χρηστών, καθένα από τα οποία διαθέτει συγκεκριμένες λειτουργίες και χαρακτηριστικά.

Το διαχειριστικό υποσύστημα παρέχει στον διαχειριστή του συστήματος τη δυνατότητα να διαχειρίζεται τις αιτήσεις εγγραφής νέων χρηστών, τα δεδομένα των εγγεγραμμένων χρηστών, τους γυμναστές, τα προγράμματα που προσφέρονται, καθώς και το πρόγραμμα των ομαδικών μαθημάτων. Επίσης, υποστηρίζει τη δημιουργία, τροποποίηση και διαγραφή ανακοινώσεων ή προσφορών που είναι ορατές στους εγγεγραμμένους χρήστες.

Το υποσύστημα χρηστών επιτρέπει στους εγγεγραμμένους χρήστες να πλοηγούνται στις διαθέσιμες υπηρεσίες του γυμναστηρίου, να κλείνουν ραντεβού για ατομικά ή ομαδικά προγράμματα και να παρακολουθούν το ιστορικό των κρατήσεών τους. Επιπλέον, οι χρήστες μπορούν να ακυρώσουν ραντεβού, υπό ορισμένες προϋποθέσεις, και να ενημερώνονται για ανακοινώσεις και νέες προσφορές.

Η υλοποίηση του συστήματος βασίστηκε σε σύγχρονες τεχνολογίες, όπως η πλατφόρμα Node.js για την ανάπτυξη της εφαρμογής, η βάση δεδομένων MongoDB για την αποθήκευση των δεδομένων και το Express.js για τη διαχείριση των αιτημάτων HTTP. Το σύστημα σχεδιάστηκε με γνώμονα την ασφάλεια και την ευκολία στη χρήση, ενώ παράλληλα εξασφαλίζει την ομαλή λειτουργία και την επεκτασιμότητα.

Το πληροφοριακό σύστημα αυτό επιτρέπει την αυτοματοποίηση πολλών διαδικασιών, όπως η έγκριση εγγραφών, η διαχείριση προγραμμάτων και γυμναστών, καθώς και η κράτηση ραντεβού, προσφέροντας έναν εύχρηστο και αποδοτικό τρόπο διαχείρισης για το προσωπικό του γυμναστηρίου και τους πελάτες του. Με την ολοκλήρωσή του, το σύστημα αποτελεί ένα ισχυρό εργαλείο για τη βελτίωση της εμπειρίας χρήστη και την ενίσχυση της αποτελεσματικότητας στη λειτουργία ενός γυμναστηρίου.

Κεφάλαιο 2: Περιγραφή συστήματος που υλοποιήθηκε

Λειτουργίες

1. Διαχειριστικό Υποσύστημα (Admin Subsystem)

Το διαχειριστικό υποσύστημα παρέχει στους διαχειριστές του γυμναστηρίου ένα σύνολο εργαλείων για την πλήρη διαχείριση των δεδομένων του συστήματος:

1. **Διαχείριση Αιτημάτων Εγγραφής Χρηστών**
   Οι διαχειριστές έχουν τη δυνατότητα να βλέπουν τα αιτήματα εγγραφής χρηστών, να τα αποδέχονται ή να τα απορρίπτουν. Κατά την αποδοχή, ανατίθεται ρόλος στον χρήστη (απλός χρήστης ή διαχειριστής).

2. **Διαχείριση Χρηστών**
   Οι διαχειριστές μπορούν να:
   - Επεξεργάζονται στοιχεία χρηστών.
   - Διαγράφουν χρήστες από το σύστημα.
   - Τροποποιούν τον ρόλο τους.

3. **Διαχείριση Γυμναστών και Προγραμμάτων**
   Το σύστημα επιτρέπει στους διαχειριστές να διαχειρίζονται:
   - **Γυμναστές**: Εισαγωγή, επεξεργασία και διαγραφή γυμναστών, μαζί με τις ειδικότητές τους.
   - **Προγράμματα**: Εισαγωγή, επεξεργασία και διαγραφή προγραμμάτων γυμναστικής.

4. **Διαχείριση Ανακοινώσεων και Προσφορών**
   Οι διαχειριστές μπορούν να δημιουργούν, να τροποποιούν και να διαγράφουν ανακοινώσεις, οι οποίες είναι ορατές μόνο στους εγγεγραμμένους χρήστες.

5. **Διαχείριση Προγραμματισμού Ομαδικών Προγραμμάτων**
   - Καταχώρηση ομαδικών προγραμμάτων με μέγιστη χωρητικότητα ατόμων, ημέρα, ώρα και υπεύθυνο γυμναστή.
   - Έλεγχος επικάλυψης ωρών και ειδικότητας γυμναστών.

2. Υποσύστημα Χρηστών (User Subsystem)

Το υποσύστημα χρηστών εξυπηρετεί τους εγγεγραμμένους χρήστες και τους επισκέπτες του συστήματος:

1. **Περιήγηση στις Υπηρεσίες του Γυμναστηρίου**
   Απλοί επισκέπτες μπορούν να δουν τις υπηρεσίες που προσφέρει το γυμναστήριο, χωρίς να απαιτείται σύνδεση.
2. **Κρατήσεις Ραντεβού**
   - Εγγεγραμμένοι χρήστες μπορούν να πραγματοποιούν κράτηση σε ατομικά ή ομαδικά προγράμματα.
   - Το σύστημα ελέγχει τη διαθεσιμότητα των θέσεων και επιτρέπει την κράτηση, εφόσον υπάρχουν διαθέσιμες θέσεις.
3. **Ακύρωση Ραντεβού**
   Οι χρήστες μπορούν να ακυρώσουν τα ραντεβού τους έως 2 ώρες πριν την έναρξή τους. Σε περίπτωση υπέρβασης ακυρώσεων, περιορίζεται προσωρινά η δυνατότητα νέων κρατήσεων.
4. **Ιστορικό Κρατήσεων**
   Οι χρήστες έχουν τη δυνατότητα να δουν το ιστορικό των κρατήσεών τους, συμπεριλαμβανομένων ακυρώσεων.
5. **Ενημέρωση μέσω Ανακοινώσεων**
   Οι εγγεγραμμένοι χρήστες μπορούν να διαβάζουν ανακοινώσεις και προσφορές που αναρτά το γυμναστήριο.

Βασικά Χαρακτηριστικά του Συστήματος

1. **Πολυεπίπεδο Σύστημα Πρόσβασης**: Το σύστημα διασφαλίζει πρόσβαση ανάλογα με τον ρόλο του χρήστη (Admin ή User).
2. **Ασφάλεια Δεδομένων**: Οι κωδικοί πρόσβασης αποθηκεύονται κρυπτογραφημένοι με χρήση της βιβλιοθήκης bcrypt.
3. **Φιλικό Περιβάλλον Χρήσης**: Παρέχεται γραφικό περιβάλλον με εύκολη πλοήγηση.

Αξιοπιστία και Διαθεσιμότητα: Το σύστημα ενσωματώνει ελέγχους για τη διαχείριση ακυρώσεων και προγραμματισμών, διασφαλίζοντας την ορθή λειτουργία.

Κεφάλαιο 3: Υλοποίηση συστήματος

1. Τεχνολογίες Backend

1.1 Node.js

Το Node.js αποτέλεσε τον βασικό runtime για την ανάπτυξη του backend. Χάρη στην ασύγχρονη φύση του, το Node.js είναι εξαιρετικά αποδοτικό για εφαρμογές που διαχειρίζονται πολλές αιτήσεις ταυτόχρονα.

1.2 Express.js

Το Express.js χρησιμοποιήθηκε ως το framework για τη δημιουργία του web server. Παρείχε:

- Routing: Διαχωρισμός των routes σε διαφορετικά αρχεία για καλύτερη οργάνωση (userRoutes.js και adminRoutes.js).
- Middleware: Για την επεξεργασία αιτήσεων, αυθεντικοποίησης και στατικής εξυπηρέτησης αρχείων.

1.3 Bcrypt

Το Bcrypt χρησιμοποιήθηκε για την ασφαλή αποθήκευση των κωδικών πρόσβασης στη βάση δεδομένων. Η κρυπτογράφηση αυξάνει την ασφάλεια έναντι επιθέσεων brute force.

1.4 Session Management

Το express-session χρησιμοποιήθηκε για τη διαχείριση των συνεδριών, επιτρέποντας την ταυτοποίησή τους καθ’ όλη τη διάρκεια της αλληλεπίδρασης με την εφαρμογή.

1.5 MongoDB

Η MongoDB αποτέλεσε τη βάση δεδομένων της εφαρμογής, όπου αποθηκεύονται πληροφορίες για χρήστες, προγράμματα, αιτήσεις εγγραφής, προγραμματισμούς, κλπ. Χρησιμοποιήθηκε η βιβλιοθήκη mongodb για τη σύνδεση και διαχείριση των δεδομένων.

1.6 Path Module

Το built-in path module της Node.js επιτρέπει τη διαχείριση διαδρομών αρχείων και φακέλων.

2. Τεχνολογίες Frontend

2.1 EJS (Embedded JavaScript Templates)

Το EJS χρησιμοποιήθηκε ως το view engine για την απόδοση δυναμικού περιεχομένου στο frontend. Παρείχε τη δυνατότητα ενσωμάτωσης δεδομένων από το backend στις HTML σελίδες.

2.2 HTML/CSS

Η HTML και CSS χρησιμοποιήθηκαν για τη δημιουργία και διαμόρφωση των σελίδων της εφαρμογής, προσφέροντας μία φιλική εμπειρία χρήστη.

2.3 Static Files

Στατικά αρχεία (CSS, εικόνες) φιλοξενήθηκαν μέσω του middleware express.static.

Επεξήγηση του app.js

```
1   const express = require('express');
2   const session = require('express-session');
3   const bcrypt = require('bcrypt'); // Για τη
4   const path = require('path'); // Χρήση του
```

- **express:** Το framework Express.js για δημιουργία διακομιστή HTTP.
- **express-session:** Για διαχείριση συνεδριών (sessions).
- **bcrypt:** Για ασφαλή αποθήκευση κωδικών πρόσβασης μέσω hashing.
- **path:** Για σωστή διαχείριση αρχείων και φακέλων.

```
5   const app = express();
6   const port = 3000;
```

```
12  app.listen(port, () => {
13      console.log('App is running at http://localhost:' + port);
14  });
```

- **Δημιουργεί μια εφαρμογή Express (app).**
- **Ο διακομιστής ακούει στην πόρτα 3000.**

```
const { MongoClient } = require('mongodb');
const uri = "mongodb://localhost:27017";
const client = new MongoClient(uri);
```

- **Χρησιμοποιεί το MongoDB client για σύνδεση στη βάση δεδομένων στη διεύθυνση mongodb://localhost:27017.**

```
app.use(session({
    secret: 'your-secret-key',  // Κλειδί για τη
    resave: false,
    saveUninitialized: true
}));
// Middleware για να διαχειριζόμαστε JSON δεδομέ
app.use(express.json());
// Για την ανάλυση της φόρμας URL encoded
app.use(express.urlencoded({ extended: true }));
```

Διαχείριση συνεδριών:

- **secret:** Μυστικό κλειδί για την κρυπτογράφηση της συνεδρίας.
- **resave: false:** Δεν αποθηκεύει τη συνεδρία αν δεν αλλάξει.
- **saveUninitialized: true:** Αποθηκεύει τις νέες συνεδρίες ακόμα και αν δεν έχουν δεδομένα.

Middleware για διαχείριση δεδομένων:

- **express.json():** Ανάλυση δεδομένων JSON από τα αιτήματα.
- **express.urlencoded({ extended: true }):** Επιτρέπει την ανάλυση URL-encoded δεδομένων (π.χ. από φόρμες).

```
app.use(express.static(path.join(__dirname, 'templates')));
app.use('/css', express.static(path.join(__dirname, 'css')));
```

- **Ορίζει τον φάκελο templates για HTML αρχεία.**
- **Ορίζει τον φάκελο css για αρχεία CSS.**

```
async function connectDB() {
    try {
        await client.connect(); // Περιμένουμε τη σύνδεση
        console.log('Connected to MongoDB');
        db = client.db('GoldenGym'); // Αποθηκεύουμε τη βάση δ
        app.locals.db = db; // Αποθήκευση του db στο app.local
        await createAdminUser(); // Κλήση για να δημιουργηθεί
    }
    catch (error) {
        console.error('Error connecting to MongoDB:', error);
    }
}
// Κλήση της συνάρτησης για να συνδεθούμε με τη βάση δεδομένων
connectDB();
```

- **Συνδέεται στη βάση δεδομένων "GoldenGym".**
- **Αποθηκεύει τη βάση στο app.locals.db, ώστε να είναι προσβάσιμη σε όλα τα routes.**
- **Καλεί τη συνάρτηση createAdminUser() για να δημιουργήσει έναν admin αν δεν υπάρχει.**

```
async function createAdminUser() {
    try {
        // Έλεγχος αν ο χρήστης admin υπάρχει ήδη
        const existingAdmin = await db.collection('users').findOne({ username: 'admin' });
        if (!existingAdmin) {
            // Κρυπτογράφηση του κωδικού του admin
            const hashedPassword = await bcrypt.hash('admin321', 10);
            // Δημιουργία χρήστη admin
            const adminUser = {
                firstName: 'Admin',
                lastName: 'Admin lastname',
                country: 'Greece',
                city: 'Athens',
                address: 'Unknown',
                email: 'admin@example.com',
                username: 'admin',
                password: hashedPassword,
                role: 'admin',
            };
            // Εισαγωγή του χρήστη στη βάση δεδομένων
            await db.collection('users').insertOne(adminUser);
            console.log('Admin user created successfully');
        }
        else {
            console.log('Admin user already exists');
        }
    }
    catch (error) {
        console.error('Error creating admin user:', error);
    }
}
```

- **Αν δεν υπάρχει admin χρήστης, δημιουργεί έναν με username: admin και password: admin321 (κρυπτογραφημένο).**

Web Services (app.js)

1. Αρχική Σελίδα

```
78 app.get('/', (req, res) => {
79     res.sendFile(path.join(__dirname, 'templates', 'index.html'));
80 });
```

- **Endpoint:** /
- **Μέθοδος:** GET
- **Λειτουργία:** Επιστρέφει το αρχικό index.html αρχείο της εφαρμογής.
- **Header:** Content-Type: text/html

2. Σελίδα Εγγραφής Χρήστη

```
83 app.get('/register', (req, res) => {
84     res.sendFile(path.join(__dirname, 'templates', 'register.html'));
85 });
```

- **Endpoint:** /register
- **Μέθοδος:** GET
- **Λειτουργία:** Επιστρέφει το register.html αρχείο για την εγγραφή χρήστη.
- **Header:** Content-Type: text/html

3. Εγγραφή Νέου Χρήστη

```
87 app.post('/register', async (req, res) => {
```

…

```
140     catch (error) {
141         console.error('Error during registration:', error);
142         res.status(500).send({ error: 'Internal server error' });
143     }
144 });
```

- **Endpoint:** /register
- **Μέθοδος:** POST
- **Λειτουργία:** Υποβάλλει αίτηση εγγραφής χρήστη, ελέγχει αν υπάρχει ήδη το email ή το username και αποθηκεύει την αίτηση στη βάση δεδομένων για έγκριση.

- **Παράμετροι:** firstName, lastName, country, city, address, email, username, password (Προέρχονται από το σώμα της αίτησης, το οποίο ανακτάται μέσω του req.body)
- **Header:** Content-Type: application/json
- **Body:** {

  “firstName”: “John”,
  “lastName”: “Doe”,
  “country”: “Greece”,
  “city”: “Athens”,
  “address”: “Somewhere”,
  “email": “john.doe@example.com”,
  “username”: “johndoe”,
  “password”: “securepassword”

  }

- **Response**
  - **201 Created** αν η αίτηση υποβληθεί με επιτυχία.
  - **400 Bad Request** αν υπάρχει ήδη χρήστης ή αίτηση με το ίδιο email/username.
  - **500 Internal Server Error** σε περίπτωση αποτυχίας του διακομιστή. (έχει χρησιμοποιηθεί σε όλα τα web services)

4. Σελίδα Σύνδεσης Χρήστη

```
147 app.get('/login', (req, res) => {
148     res.sendFile(path.join(__dirname, 'templates', 'login.html'));
149 });
```

- **Endpoint:** /login
- **Μέθοδος:** GET
- **Λειτουργία:** Επιστρέφει το login.html αρχείο για τη σύνδεση χρήστη.
- **Header:** Content-Type: text/html

5. Σύνδεση Χρήστη

```
151  app.post('/login', async (req, res) => {
152      const { username, password } = req.body;
153      if (!username || !password) {
154          req.session.user = null; // Καθαρισμός συνεδρίας
155          return res.status(400).send({ error: 'Username and password are required' });
156      }
157      try {
158          // Αναζήτηση του χρήστη στη βάση δεδομένων
159          const user = await db.collection('users').findOne({ username });
160          if (!user) {
161              req.session.user = null; // Καθαρισμός συνεδρίας
162              return res.status(401).send({ error: 'Invalid username or password' });
163          }
164          // Έλεγχος του password
165          const isPasswordValid = await bcrypt.compare(password, user.password);
166          if (!isPasswordValid) {
167              req.session.user = null; // Καθαρισμός συνεδρίας
168              return res.status(401).send({ error: 'Invalid username or password' });
169          }
170          // Αποθήκευση του χρήστη στην session
171          req.session.user = user;
172          // Έλεγχος αν ο χρήστης έχει ρόλο admin ή user
173          if (user.role === 'admin') {
174              return res.status(200).send({ message: 'Login successful', isAdmin: true });
175          }
176          else if (user.role === 'user') {
177              return res.status(200).send({ message: 'Login successful', isUser: true });
178          }
179      }
180      catch (error) {
181          console.error('Error during login:', error);
182          res.status(500).send({ error: 'Internal server error' });
183      }
184  });
```

- **Endpoint:** /login
- **Μέθοδος:** POST
- **Λειτουργία:** Το endpoint αυτό χειρίζεται τις αιτήσεις σύνδεσης χρηστών. Ελέγχει αν ο χρήστης υπάρχει στη βάση δεδομένων, αν το όνομα χρήστη και ο κωδικός είναι σωστά και αποθηκεύει τον χρήστη στη συνεδρία.
- **Παράμετροι:** username, password (req.body)
- **Header:** Content-Type: application/json
- **Body:** {
  “username”: “johndoe”,
  “password”: “securepassword”
  }

1. **Response**
    - **200 OK** αν η σύνδεση είναι επιτυχής.
    - **400 Bad Request** αν λείπουν διαπιστευτήρια.
    - **401 Unauthorized** αν τα διαπιστευτήρια είναι λανθασμένα.

6. Αποσύνδεση Χρήστη

```
187  // Route για logout
188  app.get('/logout', (req, res) => {
189      if (!req.session.user) {
190          // Αν δεν υπάρχει χρήστης στη συνεδρία, απαγορεύουμε την πρόσβαση
191          return res.status(403).send({ error: 'Access denied: You must be logged in to log out' });
192      }
193      req.session.destroy((err) => {
194          if (err) {
195              return res.status(500).send({ error: 'Failed to log out' });
196          }
197          // Ανακατεύθυνση στη αρική σελίδα μετά το logout
198          res.redirect('/');
199      });
200  });
```

- **Endpoint:** /logout
- **Μέθοδος:** GET
- **Λειτουργία:** Τερματίζει τη συνεδρία του χρήστη και τον ανακατευθύνει στην αρχική σελίδα.
- **Header:** Content-Type: text/html
- **Response**
    - Redirect στην αρχική σελίδα.
    - **403 Forbidden** αν ο χρήστης δεν είναι συνδεδεμένος.

Επεξήγηση του adminRoutes.js

```
const express = require('express');
const path = require('path');
const router = express.Router();
const bcrypt = require('bcrypt');
const { ObjectId } = require('mongodb');
```

- **express:** Εισάγεται το framework Express για τη δημιουργία του server και τη διαχείριση των διαδρομών (routes).
- **path:** Για διαχείριση αρχείων και διαδρομών.
- **router:** Δημιουργείται ένα νέο instance του Express Router, το οποίο επιτρέπει την ομαδοποίηση και διαχείριση των σχετικών διαδρομών (routes) σε ξεχωριστό module.
- **bcrypt:** Για την κρυπτογράφηση κωδικών πρόσβασης.
- **ObjectId:** Για τη σωστή διαχείριση των MongoDB Object IDs.

```
function isAdmin(req, res, next) {
    const user = req.session.user;  // Ανάκτηση του χρ
    if (user && user.role === 'admin') {
        return next();  // Αν ο χρήστης είναι admin, π
    }
    res.status(403).send('Access denied: Only administ
}
// Εφαρμογή του middleware στις διαδρομές του admin
router.use(isAdmin);
```

- **Σκοπός:** Το middleware isAdmin έχει σκοπό να ελέγχει εάν ο χρήστης που πραγματοποιεί το αίτημα είναι διαχειριστής (admin).
- **Λειτουργία:**
  - Ανακτά την πληροφορία του χρήστη από το req.session.user, δηλαδή από τη συνεδρία (session) του χρήστη.
  - Ελέγχει αν υπάρχει ο χρήστης και αν ο ρόλος του (user.role) είναι ‘admin’.
  - Αν οι παραπάνω συνθήκες πληρούνται, καλεί τη συνάρτηση next(), επιτρέποντας την περαιτέρω επεξεργασία της διαδρομής (route).
  - Αν ο χρήστης δεν είναι διαχειριστής ή δεν υπάρχει καθόλου, αποστέλλει απάντηση με κωδικό κατάστασης **403 (Forbidden)** και ένα μήνυμα σφάλματος, εμποδίζοντας έτσι την πρόσβαση.
  - **router.use(isAdmin);**

- Κάθε αίτημα που φτάνει στον router θα περάσει πρώτα από το isAdmin. Αυτό διασφαλίζει ότι μόνο χρήστες με ρόλο ‘admin’ θα έχουν πρόσβαση στις επόμενες διαδρομές που ορίζονται στο module αυτό.

Web Services (adminRoutes.js)

1. Ταμπλό διαχειριστή

```
19  router.get('/dashboard', (req, res) => {
20      const user = req.session.user; // Λήψη χρήστη από τη συνεδρ
21      res.render('adminDashboard', { username: user.username });
22  });
```

- **Endpoint:** /admin/dashboard
- **Μέθοδος:** GET
- **Λειτουργία:** Επιστρέφει την κεντρική σελίδα του διαχειριστή, η οποία εμφανίζει το όνομα χρήστη του admin.
- **Παράμετρος:** user (είναι αποθηκευμένη στη συνεδρία req.session.user και περιλαμβάνει τα δεδομένα του συνδεδεμένου χρήστη.)
- **Header:** Content-Type: text/html
- **Response:** Εμφανίζεται HTML σελίδα που αποδίδεται από το EJS template adminDashboard με παράμετρο username.

2. Προβολή Αιτήσεων Εγγραφής

```
25  router.get('/manage-registration-requests', async (req, res) => {
26      try {
27          const db = req.app.locals.db; // Πρόσβαση στο db από το app.locals
28          const registrationRequests = await db.collection('registrationRequests').find().toArray();
29          // Αν δεν υπάρχουν αιτήσεις, στείλε το μήνυμα μέσω flashMessage
30          if (registrationRequests.length === 0) {
31              return res.render('manageRegistrationRequests', {
32                  registrationRequests: [],
33                  flashMessage: { type: 'info', text: 'No registration requests found' }
34              });
35          }
36          // Αν υπάρχουν αιτήσεις, απόδωσε τις
37          return res.render('manageRegistrationRequests', { registrationRequests, flashMessage: null });
38      } catch (error) {
39          console.error('Error fetching registration requests:', error);
40          res.status(500).send('Internal Server Error');
41      }
42  });
```

- **Endpoint:** /admin/manage-registration-requests

- **Μέθοδος:** GET
- **Λειτουργία:** Ανάκτηση όλων των αιτήσεων εγγραφής από τη βάση και απόδοση της σελίδας διαχείρισης αιτήσεων (manageRegistrationRequests), με εμφάνιση flash μηνύματος σε περίπτωση που δεν βρεθούν αιτήσεις.
- **Παράμετρος:** user (είναι αποθηκευμένη στη συνεδρία req.session.user και περιλαμβάνει τα δεδομένα του συνδεδεμένου χρήστη.)
- **Header:** Content-Type: text/html
- **Response:** HTML σελίδα με την λίστα αιτήσεων και προαιρετικό flashMessage (αν δεν υπάρχουν αιτήσεις).

3. Επεξεργασία Αίτησης Εγγραφής (Αποδοχή/Απόρριψη)

```
44  // Διαδρομή για αποδοχή ή απόρριψη αιτήσεων εγγραφής
45  router.post('/manage-registration-request/:id', async (req, res) => {
46      const { action } = req.body; // 'accept' ή 'reject'
47      const { id } = req.params; // Το ID της αίτησης που θα αποδεχτεί ή θα απορριφθεί
48      const { role } = req.body;  // Λαμβάνουμε τον ρόλο από τη φόρμα
49      try {
```

...

```
88      } catch (error) {
89          console.error('Error processing registration request:', error);
90          res.status(500).send('Internal Server Error');
91      }
92  });
```

- **Endpoint:** /admin/manage-registration-request/:id
- **Μέθοδος:** POST
- **Λειτουργία:** Επεξεργάζεται μια συγκεκριμένη αίτηση εγγραφής με βάση του id. Αν το action είναι accept: δημιουργεί νέο χρήστη (με κρυπτογράφηση του κωδικού) και διαγράφει την αίτηση, ενώ αν είναι reject απλώς διαγράφει την αίτηση.
- **Παράμετρος:** id (URL παράμετρος), action, role (req.body)
- **Header:** Content-Type: application/json
- **Body:** {
  “action”: “accept”, // ή “reject”
  “role”: “user” // default: “user”
  }

- **Response:** Απόδοση του template manageRegistrationRequests με την ενημερωμένη λίστα αιτήσεων και flashMessage επιτυχίας ή σφάλματος.
  Σφάλματα:
  - **400 Bad Request** για λανθασμένο action.
  - **404 Not Found** αν δεν βρεθεί η αίτηση.

4. Προβολή Χρηστών (Διαχείριση)

```
// Διαχείριση χρηστών - Εμφάνιση όλων των χρηστών
router.get('/manage-users', async (req, res) => {
    try {
        const db = req.app.locals.db;
        const users = await db.collection('users').find().toArray();
        return res.render('manageUsers', { users, flashMessage: null});
    } catch (error) {
        console.error('Error fetching users:', error);
        res.status(500).send('Internal Server Error');
    }
});
```

- **Endpoint:** /admin/manage-users
- **Μέθοδος:** GET
- **Λειτουργία:** Ανάκτηση όλων των εγγεγραμμένων χρηστών και απόδοση της σελίδας manageUsers για διαχείριση.
- **Header:** Content-Type: text/html
- **Response:** HTML σελίδα που αποδίδεται από το template manageUsers με τη λίστα των χρηστών.

5. Φόρμα Επεξεργασίας Χρήστη

```
router.get('/edit-user/:id', async (req, res) => {
    const { id } = req.params;
    try {
        const db = req.app.locals.db;
        const user = await db.collection('users').findOne({ _id: new ObjectId(id) });
        if (!user) {
            return res.status(404).send('User not found');
        }
        return res.render('editUser', { user, flashMessage: null });
    } catch (error) {
        console.error('Error fetching user for editing:', error);
        res.status(500).send('Internal Server Error');
    }
});
```

- **Endpoint:** /admin/edit-user/:id
- **Μέθοδος:** GET
- **Λειτουργία:** Ανάκτηση δεδομένων ενός χρήστη βάση του id και απόδοση φόρμας επεξεργασίας μέσω του template editUser.
- **Header:** Content-Type: text/html
- **Παράμετροι:** id (URL παράμετρος)
- **Response:** HTML φόρμα επεξεργασίας με τα δεδομένα του χρήστη. **Σφάλμα: 404 Not Found** αν δεν βρεθεί ο χρήστης.

6. Ενημέρωση Χρήστη

```
123  // Επεξεργασία χρήστη
124  router.post('/update-user/:id', async (req, res) => {
125      const { id } = req.params;
```

...

```
185      } catch (error) {
186          console.error('Error updating user:', error);
187          res.status(500).send('Internal Server Error');
188      }
189  });
```

- **Endpoint:** /admin/update-user/:id
- **Μέθοδος:** POST
- **Λειτουργία:** Ενημερώνει τα δεδομένα ενός χρήστη με βάση το id. Ελέγχει για διπλότυπα email ή username και διασφαλίζει ότι δεν αλλάζει ο ρόλος του τελευταίου admin σε user.
- **Header:** Content-Type: application/json
- **Παράμετροι:** id (URL παράμετρος)
- **Body:** {

  “firstName”: “Όνομα”,
  “lastName”: “Επώνυμο”,
  “email”: “email@example.com”,
  “country”: “Χώρα”,
  “city”: “Πόλη”,
  “address”: “Διεύθυνση”,
  “username”: “username”,

“password”: “Νέος_κωδικός”, // Προαιρετικό: αν μείνει κενό, δεν ενημερώνεται
“role”: “admin” // ή “user”

}

- **Response:** Απόδοση της σελίδας manageUsers με ενημερωμένη λίστα χρηστών και flashMessage επιτυχίας ή σφάλματος
  Σφάλματα:
  - **400 Bad Request** για ελλιπή πεδία
  - Flash μηνύματα σε περίπτωση διπλοτύπων ή προσπάθειας αλλαγής του ρόλου του τελευταίου admin

7. Διαγραφή Χρήστη

```
// Διαγραφή χρήστη
router.post('/delete-user/:id', async (req, res) => {
    const { id } = req.params;
    try {
        const db = req.app.locals.db;
        const userToDelete = await db.collection('users').findOne({ _id: new ObjectId(id) });
        if (!userToDelete) {
            return res.status(404).send('User not found');
        }
        if (userToDelete.role === 'admin') {
            // Έλεγχος αν υπάρχει τουλάχιστον άλλος ένας διαχειριστής
            const adminCount = await db.collection('users').countDocuments({
                role: 'admin',
                _id: { $ne: new ObjectId(id) }, // Εξαίρεση του χρήστη προς διαγραφή
            });
            if (adminCount === 0) {
                const users = await db.collection('users').find().toArray();
                return res.render('manageUsers', {
                    users,
                    flashMessage: { type: 'danger', text: 'Cannot delete the last admin.' }
                });
            }
        }
        await db.collection('users').deleteOne({ _id: new ObjectId(id) });
        const users = await db.collection('users').find().toArray();
        return res.render('manageUsers', {
            users,
            flashMessage: { type: 'success', text: 'User deleted successfully!' }
        });
    } catch (error) {
        console.error('Error deleting user:', error);
        res.status(500).send('Internal Server Error');
    }
});
```

- **Endpoint:** /admin/delete-user/:id
- **Μέθοδος:** POST
- **Λειτουργία:** Διαγράφει έναν χρήστη βάσει του id. Σε περίπτωση που ο χρήστης είναι admin, ελέγχει ότι δεν πρόκειται για τον τελευταίο διαχειριστή.
- **Header:** Content-Type: text/html
- **Παράμετροι:** id (URL παράμετρος)
- **Response:** Απόδοση της σελίδας manageUsers με ενημερωμένη λίστα χρηστών και flashMessage επιτυχίας.
  Σφάλματα:
  - **404 Not Found** αν ο χρήστης δεν βρεθεί
  - Flash μηνύματα για προσπάθεια διαγραφής του τελευταίου admin.

8. Διαχείριση Δομής Συστήματος

```
227  router.get('/manage-system-structure', (req, res) => {
228      res.sendFile(path.join(__dirname, '../templates/manageSystemStructure.html'))
229  });
```

- **Endpoint:** /admin/manage-system-structure
- **Μέθοδος:** GET
- **Λειτουργία:** Στέλνει το στατικό αρχείο HTML για τη διαχείριση της δομής του συστήματος.
- **Header:** Content-Type: text/html
- **Response:** HTML αρχείο (manageSystemStructure.html).

9. Προβολή Γυμναστών (Διαχείριση)

```
232  router.get('/manage-trainers', async (req, res) => {
233      try {
234          const db = req.app.locals.db;
235          const trainers = await db.collection('trainers').find().toArray();
236          const programs = await db.collection('programs').find().toArray();
237          return res.render('manageTrainers', { trainers, programs, flashMessage: null });
238      } catch (error) {
239          console.error('Error fetching trainers:', error);
240          res.status(500).send('Internal Server Error');
241      }
242  });
```

- **Endpoint:** /admin/manage-trainers
- **Μέθοδος:** GET
- **Λειτουργία:** Ανάκτηση λίστας γυμναστών και προγραμμάτων για διαχείριση μέσω του template manageTrainers.
- **Header:** Content-Type: text/html
- **Response:** HTML σελίδα με πίνακες trainers, programs και προαιρετικό flashMessage.

10. Προσθήκη Γυμναστή

```
244  router.post('/trainers', async (req, res) => {
245      const { firstName, lastName, specialty } = req.body;
246      if (!firstName || !lastName) {
247          return res.status(400).send({ error: 'All fields are required' });
248      }
249      try {
250          const db = req.app.locals.db;
251          // Έλεγχος αν το όνομα υπάρχει ήδη
252          const programs = await db.collection('programs').find().toArray();
253          const trainers = await db.collection('trainers').find().toArray();
254          const existingTrainer = await db.collection('trainers').findOne({ firstName, lastName
255          if (existingTrainer) {
256              return res.render('manageTrainers', {
257                  trainers, programs,
258                  flashMessage: { type: 'danger', text: 'A trainer with this name already exists
259              });
260          }
261          if (!specialty || (Array.isArray(specialty) && specialty.length === 0) || (!Array.isAr
262              return res.render('manageTrainers', {
263                  trainers, programs,
264                  flashMessage: { type: 'danger', text: 'At least one speciality is required' }
265              });
266          }
267          const specialties = Array.isArray(specialty) ? specialty.slice(0, 6) : [specialty].fil
268          await db.collection('trainers').insertOne({ firstName, lastName, specialty: specialtie
269          const newTrainers = await db.collection('trainers').find().toArray();
270          return res.render('manageTrainers', {
271              trainers: newTrainers, programs,
272              flashMessage: { type: 'success', text: 'Trainer added successfully!' }
273          });
274      } catch (error) {
275          console.error('Error adding trainer:', error);
276          res.status(500).send('Internal Server Error');
277      }
278  });
```

- **Endpoint:** /admin/trainers

- **Μέθοδος:** POST
- **Λειτουργία:** Δημιουργεί νέο γυμναστή αφού ελεγχθεί ότι δεν υπάρχει ήδη με το ίδιο όνομα και ότι έχει καθορισμένη τουλάχιστον μία ειδικότητα.
- **Header:** Content-Type: application/json
- **Παράμετροι:** firstName, lastName, specialty (req.body)
- **Body:** {

  “firstName”: “Όνομα”,
  “lastName”: “Επώνυμο”,
  “specialty”: “Ειδικότητα” // ή ["Ειδικότητα1", "Ειδικότητα2"] (μέγιστος αριθμός 6)

  }
- **Response:** Απόδοση του template manageTrainers με ενημερωμένη λίστα και flashMessage επιτυχίας ή σφάλματος.

11. Ενημέρωση Γυμναστή

```
280  router.post('/trainers/update/:id', async (req, res) => {
281      const { id } = req.params;
282      const { firstName, lastName, specialty } = req.body;
283      if (!firstName || !lastName) {
284          return res.status(400).send({ error: 'All fields are required' });
285      }
```

...

```
313          return res.render('manageTrainers', {
314              trainers: updatedTrainers, programs,
315              flashMessage: { type: 'success', text: 'Trainer updated successfully!' }
316          });
317      } catch (error) {
318          console.error('Error updating trainer:', error);
319          res.status(500).send('Internal Server Error');
320      }
321  });
```

- **Endpoint:** /admin/trainers/update/:id
- **Μέθοδος:** POST

- **Λειτουργία:** Ενημερώνει τα στοιχεία ενός γυμναστή βάσει του id, αφού ελεγχθεί ότι το όνομα και το επίθετο είναι μοναδικό.
- **Header:** Content-Type: application/json
- **Παράμετροι:** id (URL), firstName, lastName, specialty (req.body)
- **Body:** {
  "firstName": "Όνομα",
  "lastName": "Επώνυμο",
  " specialty": "Νέα Ειδικότητα"
  // ή ["Ειδικότητα1", "Ειδικότητα2"] (μέγιστος αριθμός 6)
  }
- **Response:** Απόδοση του manageTrainers με την ενημερωμένη λίστα και flashMessage.
  Σφάλμα: Επιστρέφει flash error μήνυμα αν βρεθεί γυμναστής με τα ίδια στοιχεία (εκτός του τρέχοντος).

12. Διαγραφή Γυμναστή

```
323 router.post('/trainers/delete/:id', async (req, res) => {
324     const { id } = req.params;
325     try {
326         const db = req.app.locals.db;
327         const programs = await db.collection('programs').find().toArray();
328         const trainerToDelete = await db.collection('trainers').findOne({ _id: new ObjectId(i
329         if (!trainerToDelete) {
330             return res.status(404).send('Trainer not found');
331         }
332         await db.collection('trainers').deleteOne({ _id: new ObjectId(id) });
333         // Διαγραφή όλων των schedules που περιλαμβάνουν τον trainer
334         await db.collection('schedule').deleteMany({ trainerId: new ObjectId(id) });
335         const trainers = await db.collection('trainers').find().toArray();
336         return res.render('manageTrainers', {
337             trainers, programs,
338             flashMessage: { type: 'success', text: 'Trainer deleted successfully!' }
339         });
340     } catch (error) {
341         console.error('Error deleting trainer:', error);
342         res.status(500).send('Internal Server Error');
343     }
344 });
```

- **Endpoint:** /admin/trainers/delete/:id
- **Μέθοδος:** POST
- **Λειτουργία:** Διαγράφει έναν γυμναστή και, επιπλέον, όλα τα προγράμματα (schedules) στα οποία συμμετέχει.
- **Header:** Content-Type: text/html
- **Παράμετροι:** id (URL)
- **Response:** Απόδοση του template manageTrainers με ανανεωμένη λίστα και flashMessage.
  Σφάλμα: 404 Not Found αν ο trainer δεν βρεθεί

13. Προβολή Προγραμμάτων Γυμναστηρίου

```
347  // CRUD για προγράμματα
348  router.get('/manage-gym-programs', async (req, res) => {
349      try {
350          const db = req.app.locals.db;
351          const programs = await db.collection('programs').find().toArray();
352          return res.render('managePrograms', { programs, flashMessage: null });
353      } catch (error) {
354          console.error('Error fetching programs:', error);
355          res.status(500).send('Internal Server Error');
356      }
357  });
```

- **Endpoint:** /admin/manage-gym-programs
- **Μέθοδος:** GET
- **Λειτουργία:** Ανάκτηση όλων των προγραμμάτων και απόδοση της σελίδας managePrograms.
- **Header:** Content-Type: text/html
- **Response:** HTML σελίδα με τη λίστα προγραμμάτων και προαιρετικό flashMessage.

14. Προσθήκη Προγράμματος Γυμναστηρίου

```
359  router.post('/programs', async (req, res) => {
360      const { name, description } = req.body;
361      if (!name) {
362          return res.status(404).send('A program name is required');
363      }
364      try {
365          const db = req.app.locals.db;
366          const programs = await db.collection('programs').find().toArray();
367          // Έλεγχος αν το όνομα υπάρχει ήδη
368          const existingProgram = await db.collection('programs').findOne({ name: name });
369          if (existingProgram) {
370              return res.render('managePrograms', {
371                  programs,
372                  flashMessage: { type: 'danger', text: 'A program with this name already exists
373              });
374          }
375          // Προσθήκη του προγράμματος αν δεν υπάρχει
376          await db.collection('programs').insertOne({ name, description });
377          const newPrograms = await db.collection('programs').find().toArray();
378          return res.render('managePrograms', {
379              programs: newPrograms,
380              flashMessage: { type: 'success', text: 'Program added successfully!' }
381          });
382      } catch (error) {
383          console.error('Error adding program:', error);
384          res.status(500).send('Internal Server Error');
385      }
```

- **Endpoint:** /admin/programs
- **Μέθοδος:** POST
- **Λειτουργία:** Δημιουργεί νέο πρόγραμμα αφού επαληθευτεί ότι το όνομα δεν υπάρχει ήδη.
- **Header:** Content-Type: application/json
- **Παράμετροι:** name, description (req.body)
- **Body:** {
  “name”: “Όνομα Προγράμματος”,
  “description”: “Περιγραφή προγράμματος”
  // Προαιρετικό
  }
- **Response:** Απόδοση του managePrograms με ενημερωμένη λίστα και flashMessage.
  Σφάλματα: Επιστρέφει μήνυμα αν λείπει το όνομα ή αν υπάρχει ήδη πρόγραμμα με το ίδιο όνομα.

15. Ενημέρωση Προγράμματος Γυμναστηρίου

```
388  router.post('/programs/update/:id', async (req, res) => {
389      const { id } = req.params;
390      const { name, description } = req.body;
391      if (!name) {
392          return res.status(400).send({ error: 'A program name is required' });
393      }
394      try {
395          const db = req.app.locals.db;
396          const programToUpdate = await db.collection('programs').findOne({ _id: new ObjectId(id
397          if (!programToUpdate) {
398              return res.status(404).send('Program not found');
399          }
400          const programs = await db.collection('programs').find().toArray();
401          const existingProgram = await db.collection('programs').findOne({ name, _id: { $ne: ne
402          if (existingProgram) {
403              return res.render('managePrograms', {
404                  programs,
405                  flashMessage: { type: 'danger', text: 'A program with this name already exists
406              });
407          }
408          await db.collection('programs').updateOne(
409              { _id: new ObjectId(id) },
410              { $set: { name, description } }
411          );
412          const updatedPrograms = await db.collection('programs').find().toArray();
413          return res.render('managePrograms', {
414              programs: updatedPrograms,
415              flashMessage: { type: 'success', text: 'Program updated successfully!' }
416          });
417      } catch (error) {
418          console.error('Error updating program:', error);
419          res.status(500).send('Internal Server Error');
420      }
421  });
```

- **Endpoint:** /admin/programs/update/:id
- **Μέθοδος:** POST
- **Λειτουργία:** Ενημερώνει τα δεδομένα ενός προγράμματος βάσει του id.
- **Header:** Content-Type: application/json
- **Παράμετροι:** id (URL), name, description (req.body)
- **Body:** {
  “name”: “Νέο Όνομα Προγράμματος”,
  “description”: “Νέα Περιγραφή”
  }
- **Response:** Απόδοση του managePrograms με ενημερωμένη λίστα και flashMessage.
  Σφάλματα: Επιστρέφει **400** για ελλιπή πεδία ή **404** αν το πρόγραμμα δεν βρεθεί.

16. Διαγραφή Προγράμματος Γυμναστηρίου

```
423  router.post('/programs/delete/:id', async (req, res) => {
424      const { id } = req.params;
425      try {
426          const db = req.app.locals.db;
427          const programToDelete = await db.collection('programs').findOne({ _id: new ObjectId(i
428          if (!programToDelete) {
429              return res.status(404).send('Program not found');
430          }
431          await db.collection('programs').deleteOne({ _id: new ObjectId(id) });
432          const programs = await db.collection('programs').find().toArray();
433          // Διαγραφή όλων των schedules που περιλαμβάνουν το program
434          await db.collection('schedule').deleteMany({ programId: new ObjectId(id) });
435          res.render('managePrograms', {
436              programs,
437              flashMessage: { type: 'success', text: 'Program deleted successfully!' }
438          });
439      } catch (error) {
440          console.error('Error deleting program:', error);
441          res.status(500).send('Internal Server Error');
442      }
443  });
```

- **Endpoint:** /admin/programs/delete/:id
- **Μέθοδος:** POST
- **Λειτουργία:** Διαγράφει ένα πρόγραμμα και όλα τα schedule που το περιλαμβάνουν.
- **Header:** Content-Type: text/html
- **Παράμετροι:** id (URL)
- **Response:** Απόδοση του managePrograms με ενημερωμένη λίστα και flashMessage.
  Σφάλμα: Επιστρέφει **404** αν το πρόγραμμα δεν βρεθεί.

17. Προβολή Προγραμματισμών (Schedule)

```
446  // GET: Εμφάνιση προγραμματισμών
447  router.get('/manage-schedule', async (req, res) => {
448      try {
449          const db = req.app.locals.db;
450          const schedule = await db.collection('schedule').find().toArray();
451          const programs = await db.collection('programs').find().toArray();
452          const trainers = await db.collection('trainers').find().toArray();
453          return res.render('manageSchedule', { schedule, programs, trainers, flashMessage: null
454      } catch (error) {
455          console.error('Error fetching schedule:', error);
456          res.status(500).send('Internal Server Error');
457      }
458  });
```

- **Endpoint:** /admin/manage-schedule
- **Μέθοδος:** GET
- **Λειτουργία:** Ανάκτηση όλων των προγραμματισμών (schedule) μαζί με τις λεπτομέρειες προγραμμάτων και γυμναστών και απόδοση του αντίστοιχου template.
- **Header:** Content-Type: text/html
- **Response:** HTML σελίδα από το template manageSchedule με πίνακες schedule, programs, trainers και προαιρετικό flashMessage.

18. Ανάκτηση Γυμναστών Βάση Προγράμματος

```
461  router.get('/trainers-by-program', async (req, res) => {
462      const { programName } = req.query;
463      if (!programName) {
464          return res.status(400).send({ error: 'Program name is required' });
465      }
466      try {
467          const db = req.app.locals.db;
468          const trainers = await db.collection('trainers').find({
469              specialty: { $in: [programName] },
470          }).toArray();
471
472          return res.status(200).send(trainers);
473      } catch (error) {
474          console.error('Error fetching trainers:', error);
475          res.status(500).send('Internal Server Error');
476      }
477  });
```

- **Endpoint:** /admin/trainers-by-program
- **Μέθοδος:** GET
- **Λειτουργία:** Επιστρέφει μέσω API (JSON) τους γυμναστές που έχουν ως ειδικότητα το πρόγραμμα που είναι επιλεγμένο απ' τον χρήστη.
- **Header:** Content-Type: application/json
- **Παράμετροι:** programName (Query παράμετρος)
- **Response:** JSON πίνακας με τους γυμναστές.
  Σφάλμα: **400 Bad Request** αν λείπει το programName.

19. Προσθήκη Προγραμματισμού (Schedule)

```
479  // POST: Προσθήκη νέου προγραμματισμού
480  router.post('/schedule', async (req, res) => {
481      const { programName, trainerName, day, time, capacity } = req.body;
```

...

```
571      } catch (error) {
572          console.error('Error adding schedule:', error);
573          res.status(500).send('Internal Server Error');
574      }
575  });
```

- **Endpoint:** /admin/schedule
- **Μέθοδος:** POST
- **Λειτουργία:** Δημιουργεί νέο προγραμματισμό για μία συνεδρία γυμναστηρίου αφού επαληθευτούν διάφοροι έλεγχοι:
  - Η ημερομηνία και ώρα πρέπει να είναι στο μέλλον.
  - Ο γυμναστής πρέπει να έχει ως ειδικότητα το αντίστοιχο πρόγραμμα.
  - Ελέγχονται οι ώρες λειτουργίας του γυμναστηρίου (Κυριακές κλειστά, συγκεκριμένα ωράρια σε καθημερινές και Σάββατα).
  - Ελέγχεται ότι δεν υπάρχει επικαλυπτόμενος προγραμματισμός για τον γυμναστή.
- **Header:** Content-Type: application/json
- **Παράμετροι:** programName, trainerName, day, time, capacity (req.body)
- **Body:** {
  "programName": "Όνομα Προγράμματος",

“trainerName”: “Όνομα Επώνυμο”, // Πλήρες όνομα του γυμναστή
“day”: “YYYY-MM-DD”,
“time”: “HH:MM”,
“capacity”: “10”, // Αριθμητική τιμή (ως string ή αριθμός)
}

- **Response:** Απόδοση του template manageSchedule με την ανανεωμένη λίστα προγραμματισμών και flashMessage.
  Σφάλματα:
  Εμφάνιση flashMessage αν η ημερομηνία/ώρα είναι στο παρελθόν, αν είναι Κυριακή, αν δεν πληρούνται οι ώρες λειτουργίας ή αν υπάρχει επικάλυψη. Επιπλέον επιστρέφει **404 Not Found** αν δεν βρεθεί το πρόγραμμα ή ο trainer και **400** αν δοθεί trainer που δεν έχει αυτή την ειδικότητα ή για ελλιπή πεδία.

20. Ενημέρωση Προγραμματισμού (Schedule)

```
577  // POST: Ενημέρωση προγραμματισμού
578  router.post('/schedule/update/:id', async (req, res) => {
579      const { id } = req.params;
580      const { programName, trainerName, day, time, capacity } = req.body;
581      if (!programName || !trainerName || !day || !time || !capacity) {
582          return res.status(400).send({ error: 'All fields are required' });
583      }
```

…

```
681      } catch (error) {
682          console.error('Error updating schedule:', error);
683          res.status(500).send('Internal Server Error');
684      }
685  });
```

- **Endpoint:** /admin/schedule/update/:id
- **Μέθοδος:** POST
- **Λειτουργία:** Ενημερώνει έναν υπάρχοντα προγραμματισμό βάση του id με τους ίδιους ελέγχους όπως και για την προσθήκη.
- **Header:** Content-Type: application/json

- **Παράμετροι:** id (URL), programName, trainerName, day, time, capacity (req.body)
- **Body:** {

  “programName”: “Νέο Όνομα Προγράμματος”,
  “trainerName”: “Νέο Όνομα Επώνυμο”,
  “day”: “YYYY-MM-DD”,
  “time”: “HH:MM”,
  “capacity”: “10”,

  }
- **Response:** Απόδοση του manageSchedule με ανανεωμένη λίστα και flashMessage επιτυχίας ή σφάλματος.
  Σφάλματα: Επιστρέφει **404 Not Found** αν ο προγραμματισμός δεν βρεθεί ή flash μηνύματα σε περίπτωση παραβίασης κάποιου ελέγχου (και τα ίδια με το προηγούμενο).

21. Διαγραφή Προγραμματισμού (Schedule)

```
688  router.post('/schedule/delete/:id', async (req, res) => {
689      const { id } = req.params;
690      try {
691          const db = req.app.locals.db;
692          const scheduleToDelete = await db.collection('schedule').findOne({ _id: new ObjectId(i
693          if (!scheduleToDelete) {
694              return res.status(404).send('Schedule not found');
695          }
696          await db.collection('schedule').deleteOne({ _id: new ObjectId(id) });
697          const schedule = await db.collection('schedule').find().toArray();
698          const programs = await db.collection('programs').find().toArray();
699          const trainers = await db.collection('trainers').find().toArray();
700          return res.render('manageSchedule', {
701              schedule, programs, trainers,
702              flashMessage: { type: 'success', text: 'Schedule deleted successfully!' }
703          });
704      } catch (error) {
705          console.error('Error deleting schedule:', error);
706          res.status(500).send('Internal Server Error');
707      }
```

- **Endpoint:** /admin/schedule/delete/:id
- **Μέθοδος:** POST
- **Λειτουργία:** Διαγράφει έναν συγκεκριμένο προγραμματισμό βάση του id.
- **Header:** Content-Type: text/html

- **Παράμετροι:** id (URL)
- **Response:** Απόδοση του manageSchedule με την ενημερωμένη λίστα και flashMessage.
  Σφάλμα: Επιστρέφει **404 Not Found** αν ο προγραμματισμός δεν βρεθεί.

22. Προβολή Ανακοινώσεων (Διαχείριση)

```
712  router.get('/manage-announcements', async (req, res) => {
713      try {
714          const db = req.app.locals.db;
715          const announcements = await db.collection('announcements').find().toArray();
716          return res.render('ManageAnnouncements', { announcements, flashMessage: null });
717      } catch (error) {
718          console.error('Error fetching announcements:', error);
719          res.status(500).send('Internal Server Error');
720      }
721  });
```

- **Endpoint:** /admin/manage-announcements
- **Μέθοδος:** GET
- **Λειτουργία:** Ανάκτηση όλων των ανακοινώσεων και απόδοση της σελίδας ManageAnnouncements.
- **Header:** Content-Type: text/html
- **Response:** HTML σελίδα με λίστα ανακοινώσεων και προαιρετικό flashMessage.

23. Προσθήκη Ανακοίνωσης

```
724  router.post('/announcements', async (req, res) => {
725      const { title, content, validUntil } = req.body;
726      if (!title || !content || !validUntil) {
727          return res.status(400).send({ error: 'All fields are required' });
728      }
729      try {
```

...

```
756      } catch (error) {
757          console.error('Error creating announcement:', error);
758          res.status(500).send('Internal Server Error');
759      }
760  });
```

- **Endpoint:** /admin/announcements
- **Μέθοδος:** POST
- **Λειτουργία:** Δημιουργεί νέα ανακοίνωση αφού ελεγχθεί ότι έχουν δοθεί τα απαιτούμενα πεδία και ότι η ημερομηνία λήξης (validUntil) είναι στο μέλλον.
- **Header:** Content-Type: application/json
- **Παράμετροι:** title, content, validUntil (req.body)
- **Body:** {
  "title": "Τίτλος",
  "content": "Περιεχόμενο",
  "validUntil": "YYYY-MM-DD"
  }
- **Response:** Απόδοση του template ManageAnnouncements με την ενημερωμένη λίστα ανακοινώσεων και flashMessage.
  Σφάλματα: Επιστρέφει **400** αν λείπει κάποιο πεδίο και ή flashMessage σφάλματος αν η validUntil είναι στο παρελθόν.

24. Ενημέρωση Ανακοίνωσης

```
763  router.post('/announcements/update/:id', async (req, res) => {
764      const { id } = req.params;
765      const { title, content, validUntil } = req.body;
766      if (!title || !content || !validUntil) {
767          return res.status(400).send({ error: 'All fields are required' });
768      }
```

...

```
794          return res.render('ManageAnnouncements', {
795              announcements: NewAnnouncements,
796              flashMessage: { type: 'success', text: 'Announcement updated successfully!' }
797          });
798      } catch (error) {
799          console.error('Error updating announcement:', error);
800          res.status(500).send('Internal Server Error');
801      }
802  });
```

- **Endpoint:** /admin/announcements/update/:id
- **Μέθοδος:** POST

- **Λειτουργία:** Ενημερώνει τα στοιχεία μιας ανακοίνωσης βάσει του id, μετά από έλεγχο των απαιτούμενων πεδίων και της ημερομηνίας λήξης.
- **Header:** Content-Type: application/json
- **Παράμετροι:** id (URL), title, content, validUntil (req.body)
- **Body:** {
  "title": "Νέος Τίτλος",
  "content": "Νέο Περιεχόμενο",
  "validUntil": "YYYY-MM-DD"
  }
- **Response:** Απόδοση του ManageAnnouncements με την ενημερωμένη λίστα και flashMessage.
  Σφάλμα: Επιστρέφει **400** αν λείπει κάποιο πεδίο και **404 Not Found** αν η ανακοίνωση δεν βρεθεί.

25. Διαγραφή Ανακοίνωσης

```
805  router.pos }
806      const { id } = req.params;
807      try {
808          const db = req.app.locals.db;
809          const announcement = await db.collection('announcements').findOne({ _id: new Objec
810          if (!announcement) {
811              return res.status(404).send('Announcement not found');
812          }
813          await db.collection('announcements').deleteOne({ _id: new ObjectId(id) });
814          const announcements = await db.collection('announcements').find().toArray();
815          return res.render('ManageAnnouncements', {
816              announcements,
817              flashMessage: { type: 'success', text: 'Announcement deleted successfully!' }
818          });
819      } catch (error) {
820          console.error('Error deleting announcement:', error);
821          res.status(500).send('Internal Server Error');
822      }
823  });
```

- **Endpoint:** /admin/announcements/delete/:id
- **Μέθοδος:** POST
- **Λειτουργία:** Διαγράφει μια ανακοίνωση βάσει του id.
- **Header:** Content-Type: application/json
- **Παράμετροι:** id (URL)

- **Response:** Απόδοση του template ManageAnnouncements με την ενημερωμένη λίστα ανακοινώσεων και flashMessage.
  Σφάλμα: Επιστρέφει **404 Not Found** αν η ανακοίνωση δεν βρεθεί.

Επεξήγηση του userRoutes.js

```
1   const express = require('express');
2   const router = express.Router();
3   const { ObjectId } = require('mongodb');
```

- **express:** Εισάγεται το framework Express για τη δημιουργία του server και τη διαχείριση των διαδρομών (routes).
- **router:** Δημιουργείται ένα νέο instance του Express Router, το οποίο επιτρέπει την ομαδοποίηση και διαχείριση των σχετικών διαδρομών (routes) σε ξεχωριστό module.
- **ObjectId:** Για τη σωστή διαχείριση των MongoDB Object IDs.

```
5   // Ορισμός του middleware isUser
6   function isUser(req, res, next) {
7       const user = req.session.user;  // Ανάκτηση του χρήστη από τη συνεδρία
8       if (user && user.role === 'user') {
9           return next();  // Αν ο χρήστης είναι user, προχωρά στην επόμενη συνάρτηση
10      }
11      res.status(403).send('Access denied: Only users are allowed to access this resource.');
12  }
13  // Εφαρμογή του middleware στις διαδρομές του user
14  router.use(isUser);
```

- **Σκοπός:** Το middleware isUser έχει σκοπό να ελέγχει εάν ο χρήστης που πραγματοποιεί το αίτημα είναι διαχειριστής (user).
- **Λειτουργία:**
  - Ανακτά την πληροφορία του χρήστη από το req.session.user, δηλαδή από τη συνεδρία (session) του χρήστη.
  - Ελέγχει αν υπάρχει ο χρήστης και αν ο ρόλος του (user.role) είναι ‘user’.
  - Αν οι παραπάνω συνθήκες πληρούνται, καλεί τη συνάρτηση next(), επιτρέποντας την περαιτέρω επεξεργασία της διαδρομής (route).
  - Αν ο χρήστης δεν είναι απλός χρήστης ή δεν υπάρχει καθόλου, αποστέλλει απάντηση με κωδικό κατάστασης **403 (Forbidden)** και ένα μήνυμα σφάλματος, εμποδίζοντας έτσι την πρόσβαση.

- **router.use(isUser);**
  - Κάθε αίτημα που φτάνει στον router θα περάσει πρώτα από το isUser. Αυτό διασφαλίζει ότι μόνο χρήστες με ρόλο ‘user’ θα έχουν πρόσβαση στις επόμενες διαδρομές που ορίζονται στο module αυτό.

Web Services (userRoutes.js)

1. Ταμπλό Χρήστη

```
// Διαδρομή για την κεντρική σελίδα του user
router.get('/dashboard', async (req, res) => { // Δήλωση της συνάρτησης ως async
    try {
        const user = req.session.user; // Λήψη χρήστη από τη συνεδρία
        const db = req.app.locals.db; // Πρόσβαση στο db από το app.locals
        const today = new Date();
        const programs = await db.collection('programs').find().toArray();
        const announcements = await db.collection('announcements')
            .find({ validUntil: { $gte: today } })
            .sort({ validUntil: 1 })
            .toArray(); // Μετατροπή σε array για χρήση
        res.render('userDashboard', { username: user.username, announcements, programs: progra
    } catch (error) {
        console.error('Error fetching announcements:', error);
        res.status(500).send('Internal Server Error');
    }
});
```

- **Endpoint:** /user/dashboard
- **Μέθοδος:** GET
- **Λειτουργία:** Ανάκτηση δεδομένων για την κεντρική σελίδα του χρήστη. Συγκεκριμένα, φορτώνει:
  - Τη λίστα των προγραμμάτων (από τη συλλογή programs)
  - Τις ανακοινώσεις (από τη συλλογή announcements) όπου η ημερομηνία λήξης (validUntil) είναι ίση ή μεταγενέστερη της σημερινής ημερομηνίας.
  - Το username του χρήστη (από τη συνεδρία).
- **Header:** Content-Type: text/html
- **Response:** Απόδοση του userDashboard με το username απ’ το session, τις ενεργές ανακοινώσεις και τα προγράμματα.

2. Προβολή Προφίλ Χρήστη

```
34  // Διαδρομή για επιστροφή των στοιχείων του χρήστη χωρίς password και role
35  router.get('/profile', async (req, res) => {
36      try {
37          const user = req.session.user; // Λήψη του χρήστη από τη συνεδρία
38          const db = req.app.locals.db; // Πρόσβαση στη βάση δεδομένων
39          if (!user) {
40              return res.status(401).send('Unauthorized: No user logged in');
41          }
42          const userProfile = await db.collection('users').findOne(
43              { username: user.username }, // Κριτήριο αναζήτησης
44              { projection: { password: 0, role: 0 } } // Εξαίρεση των πεδίων
45          );
46          if (!userProfile) {
47              return res.status(404).send('User not found');
48          }
49          res.render('profile', { user: userProfile }); // Απόδοση του EJS vi
50      } catch (error) {
51          console.error('Error fetching user profile:', error);
52          res.status(500).send('Internal Server Error');
53      }
54  });
```

- **Endpoint:** /user/profile
- **Μέθοδος:** GET
- **Λειτουργία:** Ανάκτηση των στοιχείων του χρήστη από τη βάση δεδομένων (χωρίς τα πεδία password και role) και απόδοση του προφίλ του μέσω του view profile.
- **Header:** Content-Type: text/html
- **Response:** με ένα αντικείμενο που περιέχει τα δεδομένα του χρήστη (π.χ. username, firstName, lastName, email κ.λπ.).
  Σφάλματα:
  - **401 Unauthorized** εάν δεν υπάρχει χρήστης στη συνεδρία.
  - **404 Not Found** εάν δεν βρεθεί ο χρήστης στη βάση.

3. Προβολή Υπηρεσιών

```
56  // Διαδρομή για προγράμματα
57  router.get('/services', async (req, res) => {
58      try {
59          const db = req.app.locals.db;
60          const user = req.session.user;
61          const programs = await db.collection('programs').find().toArray();
62          const appointments = await db.collection('appointments').find({username: user.usernam
63          return res.render('services', { availableSessions: [], appointments, programs, flashM
64      } catch (error) {
65          console.error('Error fetching programs:', error);
66          res.status(500).send('Internal Server Error');
67      }
68  });
```

- **Endpoint:** /user/services
- **Μέθοδος:** GET
- **Λειτουργία:** Ανάκτηση των διαθέσιμων προγραμμάτων και των ήδη κλεισμένων ραντεβού του χρήστη (με status "booked").
- **Header:** Content-Type: text/html
- **Response:** To view services αποδίδεται με τα εξής δεδομένα:
  {
  availableSessions: [], // Αρχικά κενός πίνακας
  appointments: [ραντεβού του χρήστη με status "booked"],
  programs: [λίστα προγραμμάτων],
  flashMessage: null
  }

4. Έλεγχος Διαθεσιμότητας Συνεδριών

```
70  router.get('/availability', async (req, res) => {
71      const { programId, date } = req.query;
72      if (!programId || !date) {
73          return res.status(400).send('Program ID and Date are required');
74      }
```

...

```
104     } catch (error) {
105         console.error('Error fetching availability:', error);
106         res.status(500).send('Internal Server Error');
107     }
108 });
```

- **Endpoint:** /user/availability
- **Μέθοδος:** GET
- **Λειτουργία:** Επιστρέφει τις διαθέσιμες συνεδρίες (sessions) για ένα συγκεκριμένο πρόγραμμα και ημερομηνία.
  Ο έλεγχος περιλαμβάνει:
  - Επαλήθευση ότι έχουν δοθεί το programId και το date (σε μορφή YYYY-MM-DD) μέσω των query parameters.
  - Εάν η επιλεγμένη ημερομηνία είναι η σημερινή, προστίθεται ένας επιπλέον έλεγχος για την ώρα (μηχανισμός ώστε να μην επιστραφούν συνεδρίες που έχουν ήδη ξεκινήσει).
  - Επιστροφή του view services με τα αποτελέσματα και σε περίπτωση που δεν υπάρχουν διαθέσιμες συνεδρίες, επιστρέφεται flash μήνυμα προειδοποίησης.
- **Header:** Content-Type: text/html
- **Παράμετροι:** programId, date (Query)
- **Response:** Το view services αποδίδεται με τα εξής δεδομένα:
  {
  availableSessions: [συνεδρίες που πληρούν το query],
  appointments: [ραντεβού του χρήστη με status "booked"],
  programs: [λίστα προγραμμάτων],
  flashMessage: { type: 'warning', text: 'No available sessions for the selected date.' } // σε περίπτωση που δεν υπάρχει διαθεσιμότητα
  }
  Σφάλματα:
  - **400 Bad Request** εάν λείπουν οι απαιτούμενες παράμετροι.
  - **404 Not Found** εάν δεν βρεθεί το πρόγραμμα.

5. Κλείσιμο Ραντεβού

```
110  router.post('/book-appointment', async (req, res) => {
111      const { sessionId } = req.body;
112      const userId = req.session.user?._id;
113      if (!sessionId || !userId) {
114          return res.status(400).send('Session ID and User ID are required');
115      }
```

…

```
200      } catch (error) {
201          console.error('Error booking appointment:', error);
202          res.status(500).send('Error processing your booking');
203      }
204  });
```

- **Endpoint:** /user/book-appointment
- **Μέθοδος:** POST
- **Λειτουργία:** Κλείνει ραντεβού για μια συνεδρία (session) του γυμναστηρίου για τον τρέχοντα χρήστη, μετά από διάφορους ελέγχους: Οι έλεγχοι περιλαμβάνουν:
    - Επαλήθευση ύπαρξης του χρήστη και της συνεδρίας.
    - Εάν ο χρήστης δεν έχει υπερβεί τα όρια ακυρώσεων της εβδομάδας.
    - Εάν ο χρήστης έχει ήδη κλείσει ραντεβού για το πρόγραμμα.
    - Εάν υπάρχει διαθέσιμη θέση (capacity > 0), μειώνεται η χωρητικότητα κατά 1 και καταχωρείται το ραντεβού.
- **Header:** Content-Type: application/json
- **Παράμετροι:** sessionId (req.body), userId (από την συνεδρία)
- **Body: {**
  "sessionId": "id της συνεδρίας"
  }
- **Response:** Σε περίπτωση επιτυχίας, αποδίδεται το view services με ενημερωμένη λίστα διαθέσιμων συνεδριών και ραντεβού, καθώς και flash μήνυμα επιτυχίας:
  {
  availableSessions: [ενημερωμένος πίνακας συνεδριών],
  appointments: [ενημερωμένος πίνακας κλεισμένων ραντεβού],
  programs: [λίστα προγραμμάτων],
  flashMessage: { type: 'success', text: 'Appointment booked successfully!' }
  }
  Σφάλματα:
    - **400 Bad Request** εάν λείπει το sessionId ή το userId.
    - **404 Not Found** εάν δεν βρεθεί ο χρήστης ή η συνεδρία.
    - **409 Conflict** εάν δεν υπάρχουν διαθέσιμες θέσεις (capacity 0).
    - Επιστρέφει επίσης flash μήνυμα αν ο χρήστης έχει ήδη κλείσει ραντεβού για το συγκεκριμένο πρόγραμμα ή έχει ξεπεράσει το όριο ακυρώσεων.

6. Ακύρωση Ραντεβού

```
207 router.post('/cancel-appointment', async (req, res) => {
208     const { appointmentId } = req.body;
209     const userId = req.session.user?._id;
210     if (!appointmentId || !userId) {
211         return res.status(400).send('Appointment ID and User ID are required');
212     }
```

...

```
274     } catch (error) {
275         console.error('Error cancelling appointment:', error);
276         res.status(500).send('Error processing your cancellation');
277     }
278 });
```

- **Endpoint:** /user/cancel-appointment
- **Μέθοδος:** POST
- **Λειτουργία:** Ακυρώνει ένα κλεισμένο ραντεβού του χρήστη, υπό τους εξής ελέγχους:
    - Επαλήθευση ότι το ραντεβού υπάρχει και ανήκει στον χρήστη.
    - Εξασφάλιση ότι η ακύρωση γίνεται τουλάχιστον 2 ώρες πριν την έναρξη της συνεδρίας.
    - Καταγραφή της ακύρωσης στη συλλογή cancellations και ενημέρωση της δυνατότητας κλεισίματος νέου ραντεβού (canBook) βάση του αριθμού ακυρώσεων μέσα στην ίδια εβδομάδα.
    - Ενημέρωση του status του ραντεβού σε "cancelled" και αποκατάσταση της χωρητικότητας της συνεδρίας.
- **Header:** Content-Type: application/json
- **Παράμετροι:** appointmentId (req.body), userId (από την συνεδρία)
- **Body:** {
  “ appointmentId”: “id του ραντεβού”
  }
- **Response:** Σε περίπτωση επιτυχίας, αποδίδεται το view services με ενημερωμένο πίνακα ραντεβού και flash μήνυμα επιτυχίας:
  {
  availableSessions: [] // Ενδεχομένως κενός πίνακας,
  appointments: [ενημερωμένος πίνακας κλεισμένων ραντεβού],
  programs: [λίστα προγραμμάτων],

flashMessage: { type: 'success', text: 'Appointment cancelled successfully.' }
}

Σφάλματα:

- **400 Bad Request** εάν λείπει το sessionId ή το userId.
- **404 Not Found** εάν δεν βρεθεί ο χρήστης ή η συνεδρία.
- **409 Conflict** εάν δεν υπάρχουν διαθέσιμες θέσεις (capacity 0).

7. Προβολή Ιστορικό Κρατήσεων

```
281 router.get('/booking-history', async (req, res) => {
282     try {
283         const db = req.app.locals.db;
284         const user = req.session.user;
285         const cancellationDate = await db.collection('cancellations').find({username: user.username}).toArray();
286         const appointments = await db.collection('appointments').find({username: user.username}).toArray();
287         return res.render('bookingHistory', { appointments, cancellationDate });
288     } catch (error) {
289         console.error('Error fetching booking history:', error);
290         res.status(500).send('Internal Server Error');
291     }
292 });
```

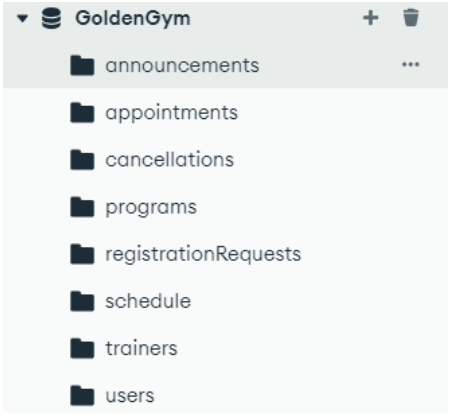
- **Endpoint:** /user/booking-history
- **Μέθοδος:** GET
- **Λειτουργία:** Ανάκτηση του ιστορικού κρατήσεων του χρήστη, το οποίο περιλαμβάνει:
  - Όλα τα ραντεβού (είτε κλεισμένα είτε ακυρωμένα) από τη συλλογή appointments.
  - Τυχόν δεδομένα ακυρώσεων από τη συλλογή cancellations.
- **Header:** Content-Type: text/html
- **Response:** Το view bookingHistory αποδίδεται με τους πίνακες appointments και cancellationDate.

Σχήμα της Βάσης Δεδομένων (MongoDB)

Ακολουθούν μερικοί λόγοι για τους οποίους χρησιμοποίησα την MongoDB:

- **Ευελιξία και Schema-less Δομή:** Η MongoDB αποθηκεύει δεδομένα σε μορφή JSON-like (BSON), επιτρέποντας δυναμικές και ευέλικτες δομές δεδομένων. Δεν απαιτεί προκαθορισμένα σχήματα (schemas), γεγονός που διευκολύνει την ανάπτυξη και τις αλλαγές στο μοντέλο δεδομένων καθώς εξελίσσεται η εφαρμογή.
- **Εύκολη Ενσωμάτωση με Node.js:** Η MongoDB λειτουργεί άψογα σε περιβάλλοντα JavaScript, καθώς οι εγγραφές της έχουν μορφή JSON, διευκολύνοντας την ενσωμάτωση με πλαίσια όπως το Express και την ανάπτυξη εφαρμογών με Node.js.
- **Απόδοση και Ταχύτητα:** Η MongoDB επιτυγχάνει υψηλή απόδοση τόσο σε ανάγνωση όσο και σε εγγραφή δεδομένων χάρη στο απλό, document-oriented μοντέλο της και τη χρήση indexing, καθιστώντας την κατάλληλη για εφαρμογές που απαιτούν γρήγορες και αποτελεσματικές λειτουργίες.
- **Ευκολία Ανάπτυξης και Απλότητα:** Η απουσία αυστηρών δομών και η φυσική διαχείριση δεδομένων με JSON επιταχύνουν την ανάπτυξη και επιτρέπουν γρήγορη προσαρμογή σε νέες απαιτήσεις χωρίς την ανάγκη πολύπλοκων αλλαγών στη βάση δεδομένων.

Η Βάση Δεδομένων ονομάζεται **GoldenGym** και περιλαμβάνει τα εξής collections:



Παραδείγματα με το τι μπορεί να περιέχει κάθε collection:

announcements:

```
_id: ObjectId('677134ab7efe5efe4414d08b')
title : "3μηνο Reformer Pilates"
content : "-50$ + Δώρο ο Γενάρης για νέα μελη!!
          Σύνολο 250$"
validUntil : 2025-02-01T00:00:00.000+00:00
createdAt : 2024-12-29T11:38:19.350+00:00
```

```
_id: ObjectId('677134d07efe5efe4414d08c')
title : "20 Personal "
content : "+5 δώρο!!
          Σύνολο 300$"
validUntil : 2025-03-13T00:00:00.000+00:00
createdAt : 2024-12-29T11:38:56.717+00:00
```

Σε αυτό το collection αποθηκεύονται οι ανακοινώσεις με το περιεχόμενο τους καθώς και την προθεσμία τους (και πότε δημιουργήθηκαν).

appointments:

```
_id: ObjectId('67a61fc52da9a1a300618884')
sessionId : ObjectId('67742c725a5bc04e4c58802f')
username : "panoschon"
programName : "Reformer Pilates"
trainerName : "Βασιλική Κόλλια"
day : "2025-03-13"
time : "19:39"
status : "booked"
```

(ή status: “cancelled”)

Εδώ αποθηκεύονται τα ραντεβού του κάθε χρήστη είτε ακυρωμένα είτε ενεργά.

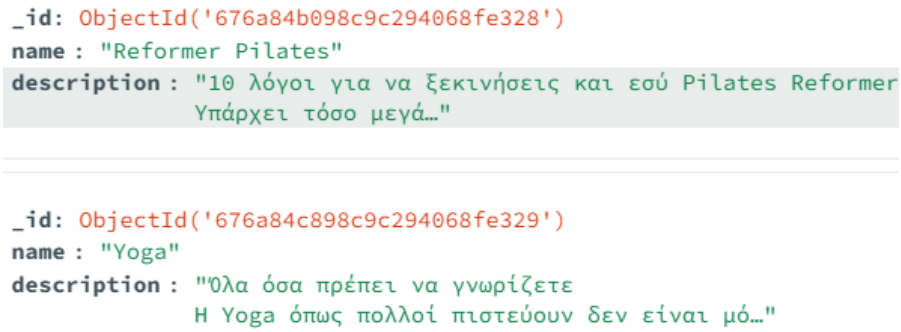
cancellations:

```
_id: ObjectId('67a620382da9a1a300618885')
appointmentId : ObjectId('67a61fc52da9a1a300618884')
username : "panoschon"
dateOfCancellation : 2025-02-07T15:01:12.261+00:00
canBook : true
```

(ή canBook: “false”)

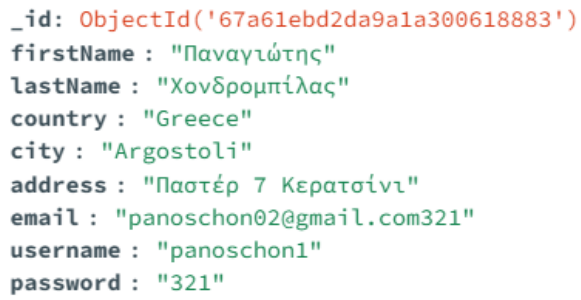
Εδώ αποθηκεύονται όλες οι ακυρώσεις του χρήστη μαζί με την ημερομηνία που ακυρώθηκε. Εάν υπερβεί τις 2 ακυρώσεις σε μία εβδομάδα τότε το canBook λαμβάνει την τιμή false απαγορεύοντας του να κλείσει άλλο ραντεβού για την συγκεκριμένη βδομάδα.

programs:



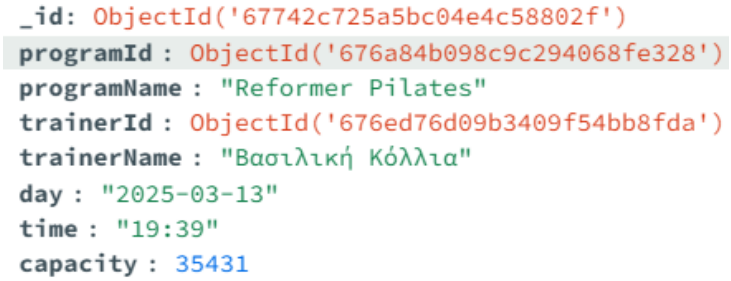
Αποθήκευση προγραμμάτων.

registrationRequests:



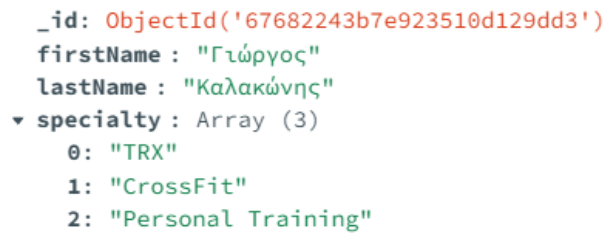
Αποθήκευση αιτήσεων εγγραφής.

schedule:



Αποθήκευση προγραμματισμών του γυμναστηρίου.

trainers:



Αποθήκευση γυμναστών με τις ιδιότητες τους.

users:



Αποθήκευση όλων των χρηστών (διαχειριστές ή απλοί χρήστες) του συστήματος με τα στοιχεία τους (κρυπτογραφημένος κωδικός).

Κεφάλαιο 4: Εγχειρίδιο διαχειριστή

Ακολουθήστε τα παρακάτω βήματα για να εγκαταστήσετε και να ρυθμίσετε το σύστημα στον υπολογιστή σας:

1. **Απαιτήσεις**

Πριν ξεκινήσετε, βεβαιωθείτε ότι ο υπολογιστής σας διαθέτει τα παρακάτω:

- **Node.js και nmp:**
  Η εφαρμογή έχει αναπτυχθεί με Node.js, γι' αυτό χρειάζεται να εγκαταστήσετε την πιο πρόσφατη έκδοση του Node.js μαζί με το npm (Node Package Manager).
  Κατέβασμα και εγκατάσταση: Node.js Official Website
- **MongoDB (Compass):**
  Η βάση δεδομένων που χρησιμοποιείται είναι η MongoDB, η οποία πρέπει να είναι εγκατεστημένη και σε λειτουργία.
  Κατέβασμα και εγκατάσταση: MongoDB Download Center

2. **Προετοιμασία του κώδικα**
   - Κατεβάστε το zip αρχείο του κώδικα και αποσυμπιέστε το στον υπολογιστή σας.

3. **Εγκατάσταση Εξαρτήσεων**
   - Για να εγκαταστήσετε όλες τις απαραίτητες βιβλιοθήκες και modules που χρησιμοποιούνται στο project, εκτελέστε μέσα στον φάκελο που είναι όλα τα αρχεία (με cmd) την εντολή: **npm install**

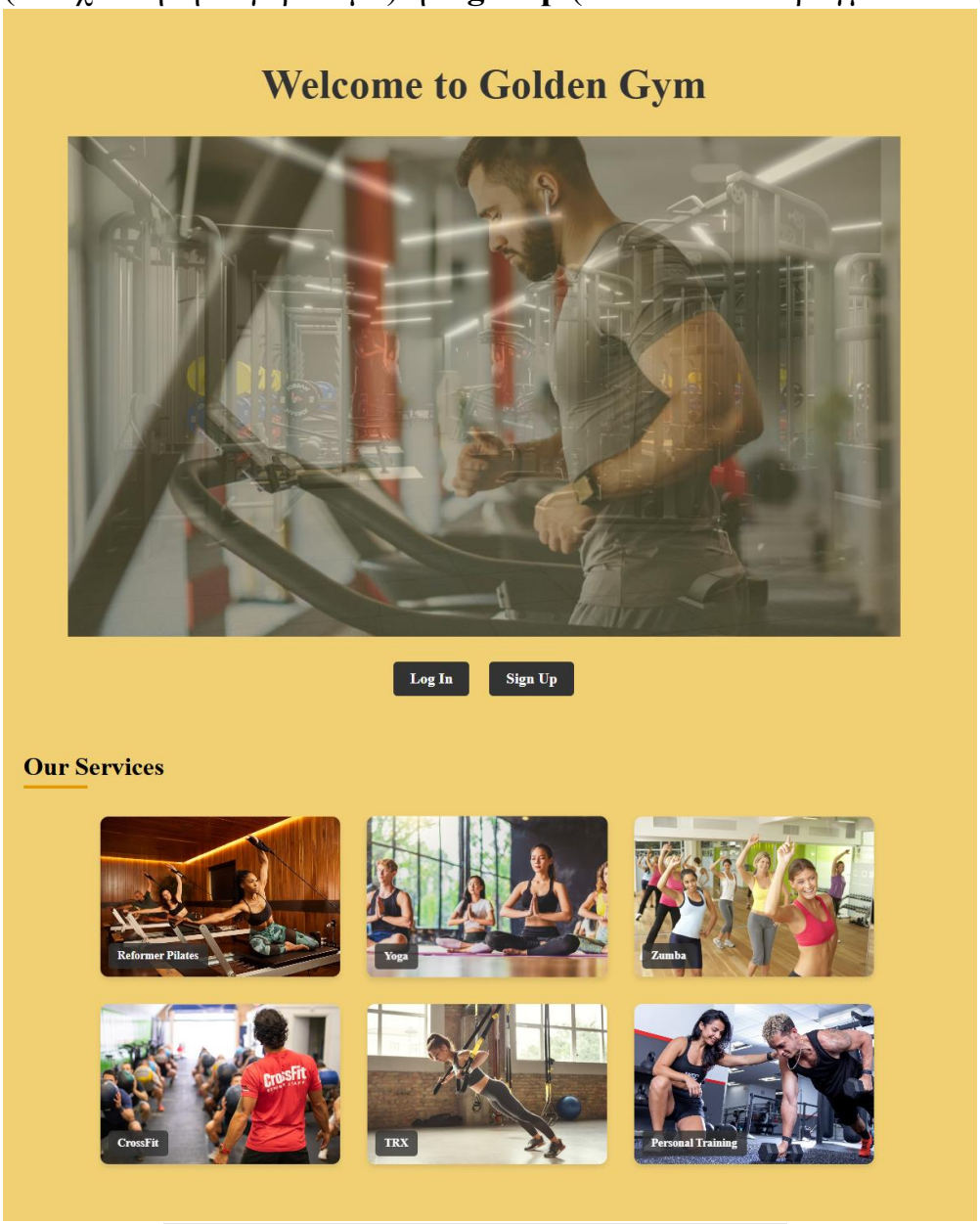
4. **Ρύθμιση της Σύνδεσης με τη MongoDB**
   - **Τοπική MongoDB:**
     Βεβαιωθείτε ότι το MongoDB τρέχει στον τοπικό σας server (στη διεύθυνση mongodb://localhost:27017).

- **Δημιουργία Βάσης Δεδομένων:**
  Ο κώδικας του συστήματος χρησιμοποιεί τη βάση δεδομένων με όνομα GoldenGym. Κατά την πρώτη εκτέλεση, το σύστημα θα συνδεθεί και θα δημιουργήσει αυτόματα την απαιτούμενη βάση δεδομένων, καθώς και τον admin, αν δεν υπάρχει.

5. **Εκτέλεση της Εφαρμογής**
   - Μετά την ολοκλήρωση των παραπάνω βημάτων, ξεκινήστε την εφαρμογή εκτελώντας: **node app.js** (μέσα στον φάκελο που είναι όλα τα αρχεία (cmd))
   - Η εφαρμογή θα ξεκινήσει στο καθορισμένο port (3000). Ανοίξτε τον περιηγητή σας και επισκεφθείτε τη διεύθυνση: **http://localhost:3000**

Κεφάλαιο 5: Εγχειρίδιο χρήστη

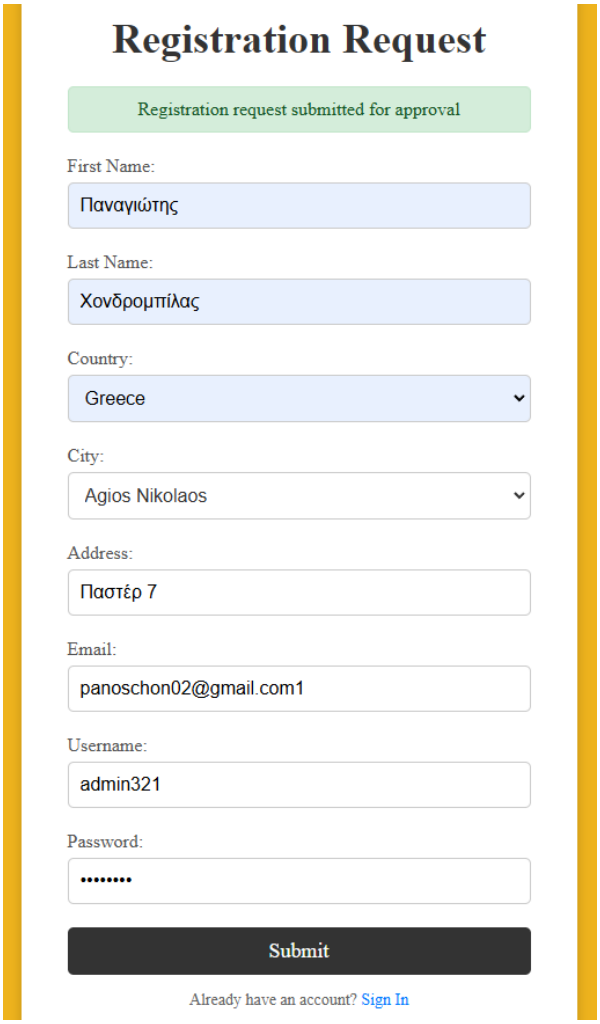
Αφού επισκεφθείτε την διεύθυνση http://localhost:3000 έχετε 2 επιλογές **Log In** (αν έχετε ήδη λογαριασμό) ή **Sign Up** (αν θέλετε να πραγματοποιήσετε εγγραφή).



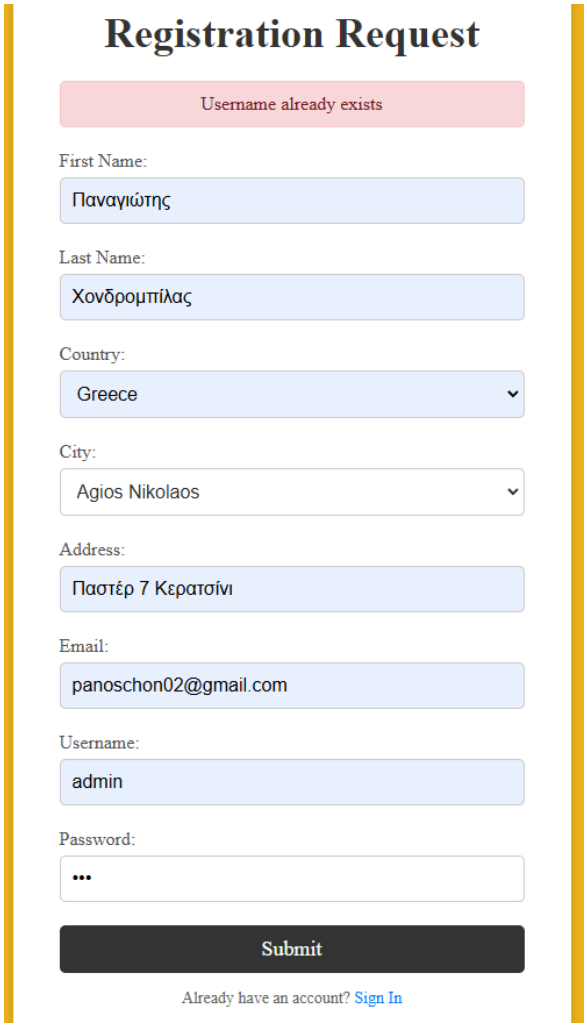


(Πατώντας το κουμπί ή στην διεύθυνση http://localhost:3000/register.html)

Sign Up: (επιτυχής αίτημα εγγραφής)

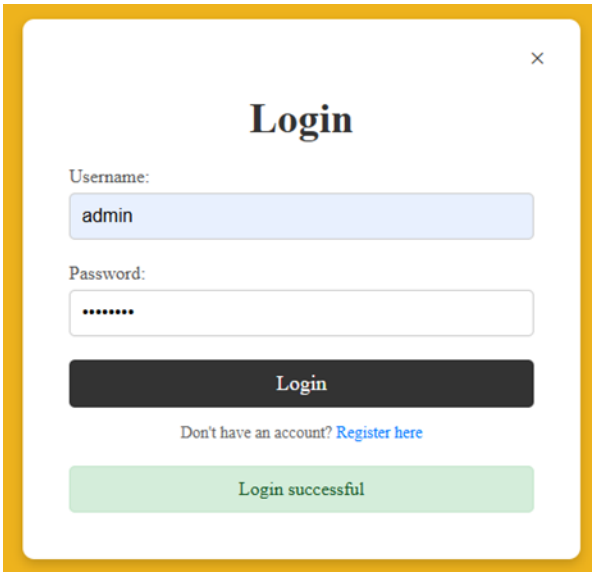


(αποτυχία αιτήματος εγγραφής)

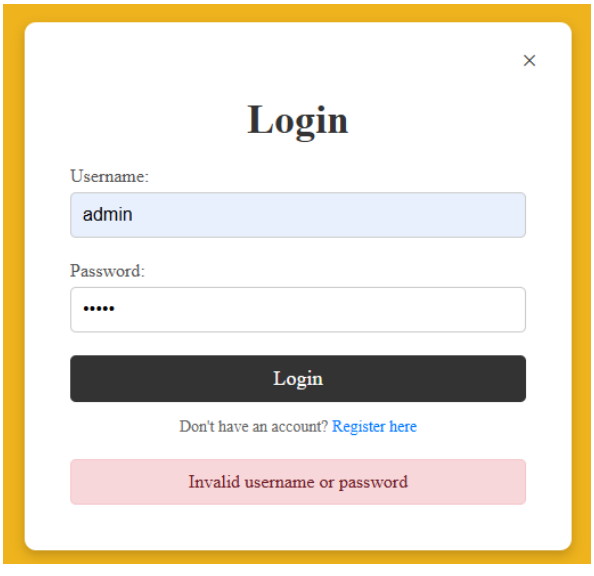


(Πατώντας το κουμπί ή στην διεύθυνση http://localhost:3000/login.html)

Log In: (επιτυχής σύνδεση)



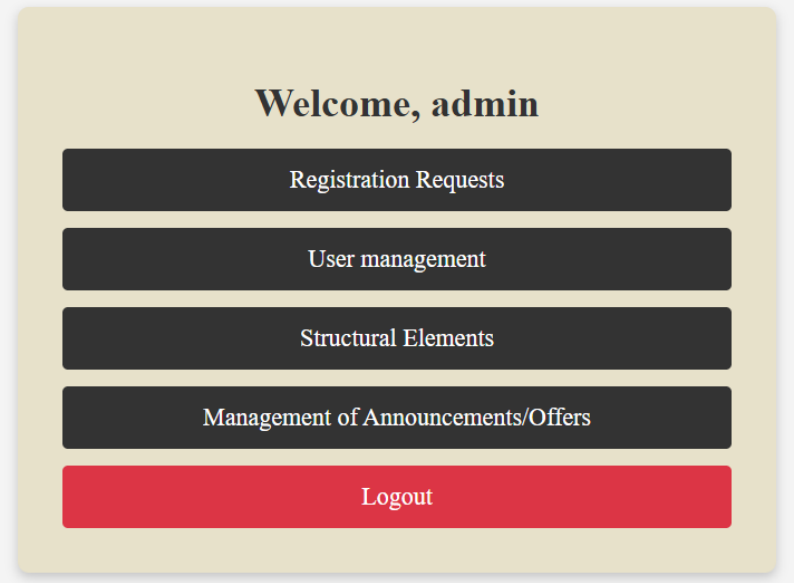
(αποτυχία σύνδεσης)



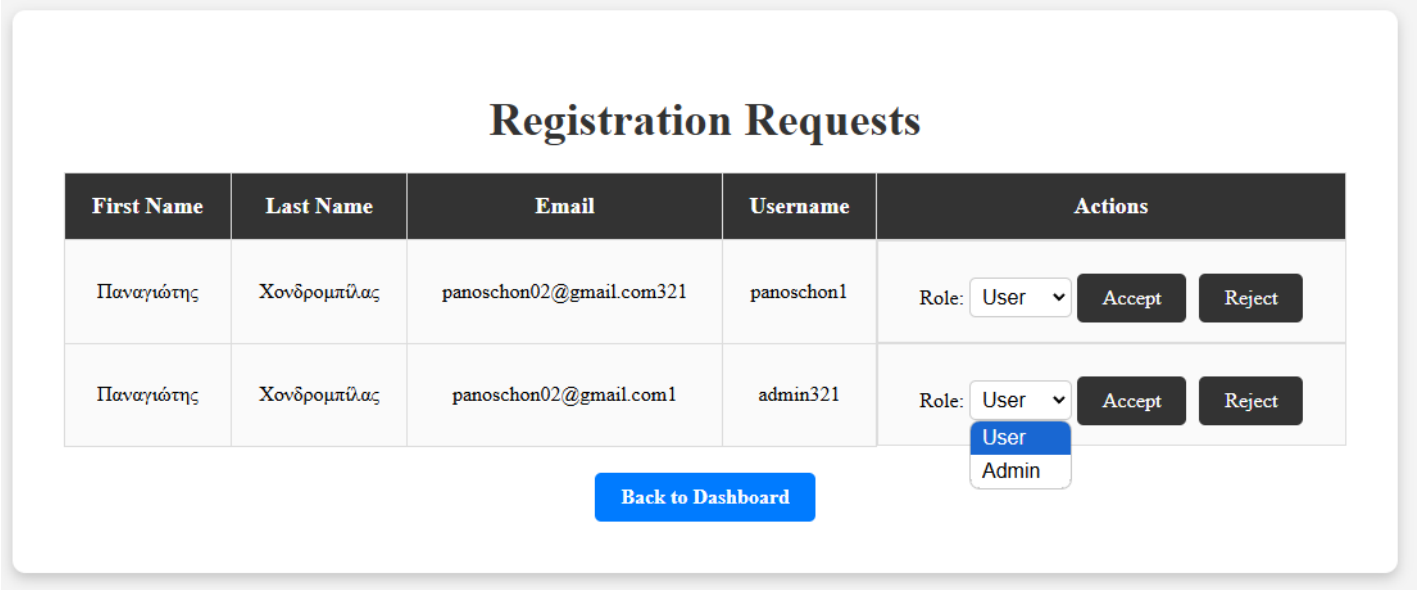
Υπάρχει σύνδεσμος που σας στέλνει απευθείας απ' το **Register** στο **Log In** (Already have an account? **Sing In**) και το αντίστροφο (Don't have an account? **Register here**).

Admin

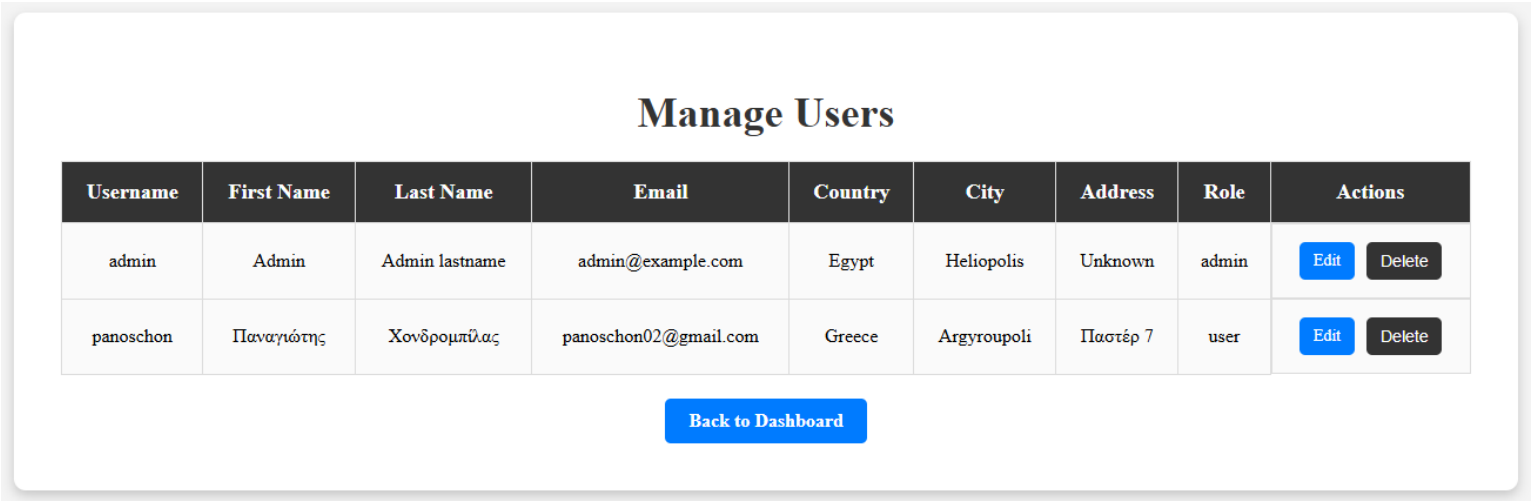
Αφού συνδεθείτε ως διαχειριστής ανακατευθύνεστε στην διεύθυνση http://localhost:3000/admin/dashboard



Πατώντας το κουμπί **Registration Requests** ανοίγει η ιστοσελίδα http://localhost:3000/admin/manage-registration-requests, όπου μπορείτε να δείτε τα αιτήματα εγγραφής και να τα εγκρίνεται ή να τα απορρίψετε. Σε περίπτωση που θέλετε να εγκρίνεται κάποιο, πρέπει να επιλέξετε το ρολό του χρήστη (user ή admin). Πατήστε **Back to Dashboard** για να επιστρέψετε στο ταμπλό του διαχειριστή (http://localhost:3000/admin/dashboard)

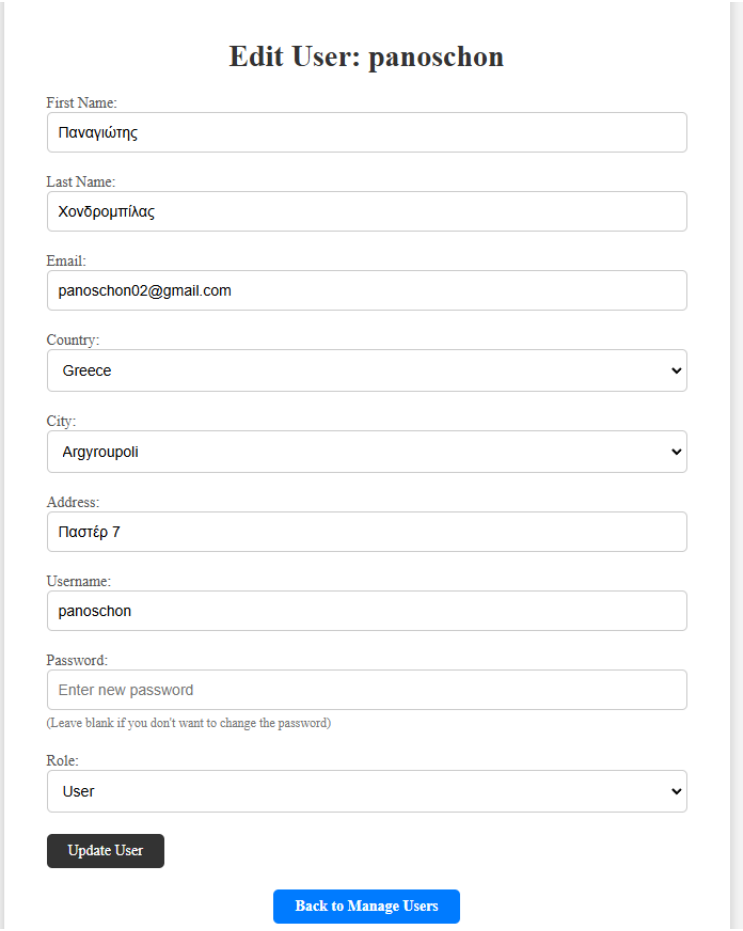


Πατώντας το κουμπί **User management** ανοίγει η ιστοσελίδα http://localhost:3000/admin/manage-users



Εδώ εμφανίζονται οι ήδη εγγεγραμμένοι χρήστες και έχετε την δυνατότητα να επεξεργαστείτε τα δεδομένα τους πατώντας το κουμπί **Edit** (**http://localhost:3000/admin/edit-user/67672e89791e52387cb1b2c7**) ή να τους διαγράψετε απ' το σύστημα με το κουμπί **Delete.**

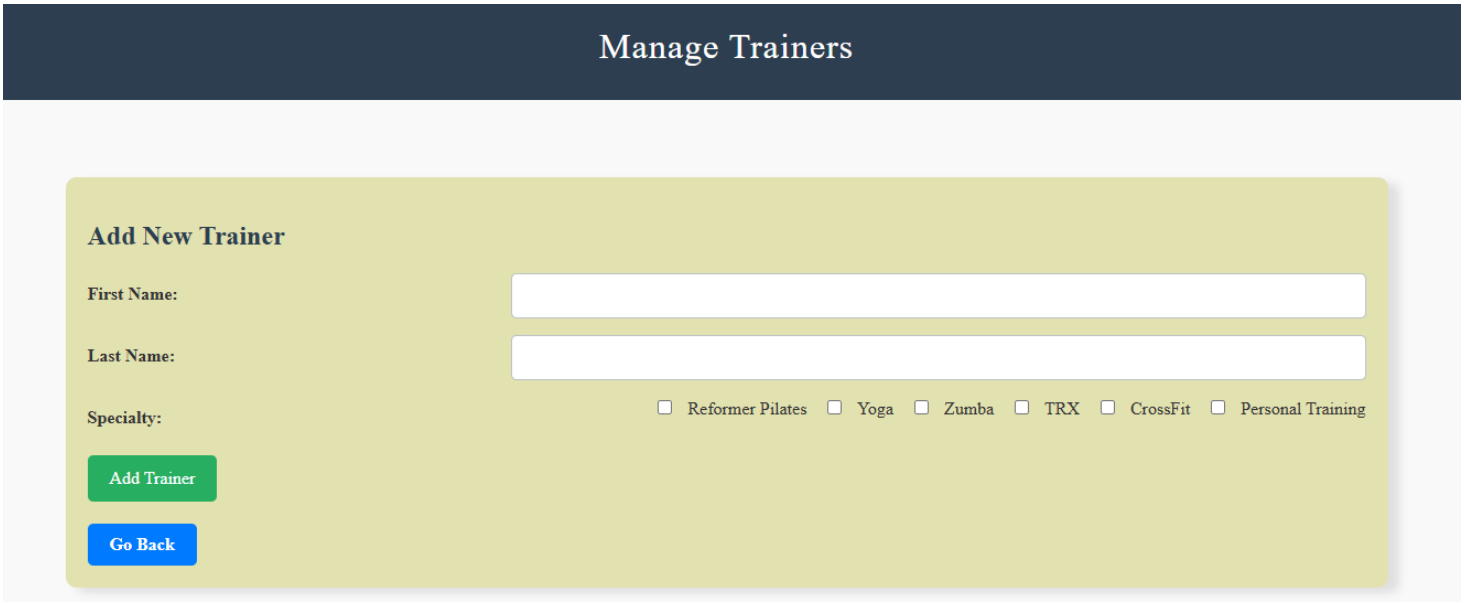
Σε αυτή στην σελίδα κάντε οποιαδήποτε αλλαγή θέλετε και επιλέξτε το κουμπί **Update User** για να επιβεβαιώσετε την ενημέρωση των στοιχείων του χρήστη. Πατήστε το κουμπί **Back to Manage Users** για να επιστρέψετε στην προηγούμενη σελίδα.



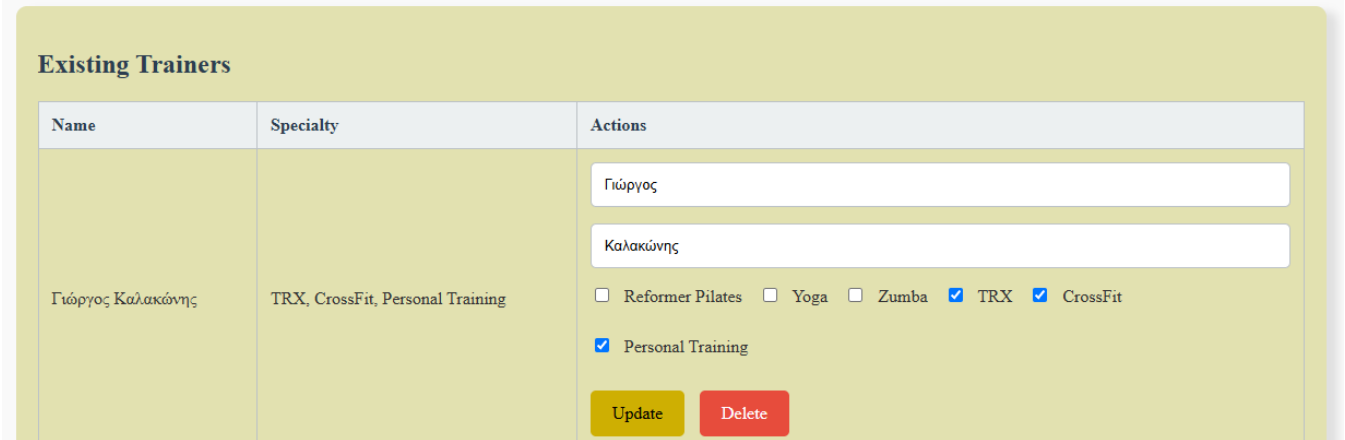
Επιλέγοντας το κουμπί **Structural Elements** ανοίγει ένα νέο ταμπλό στην διεύθυνση http://localhost:3000/admin/manage-system-structure.



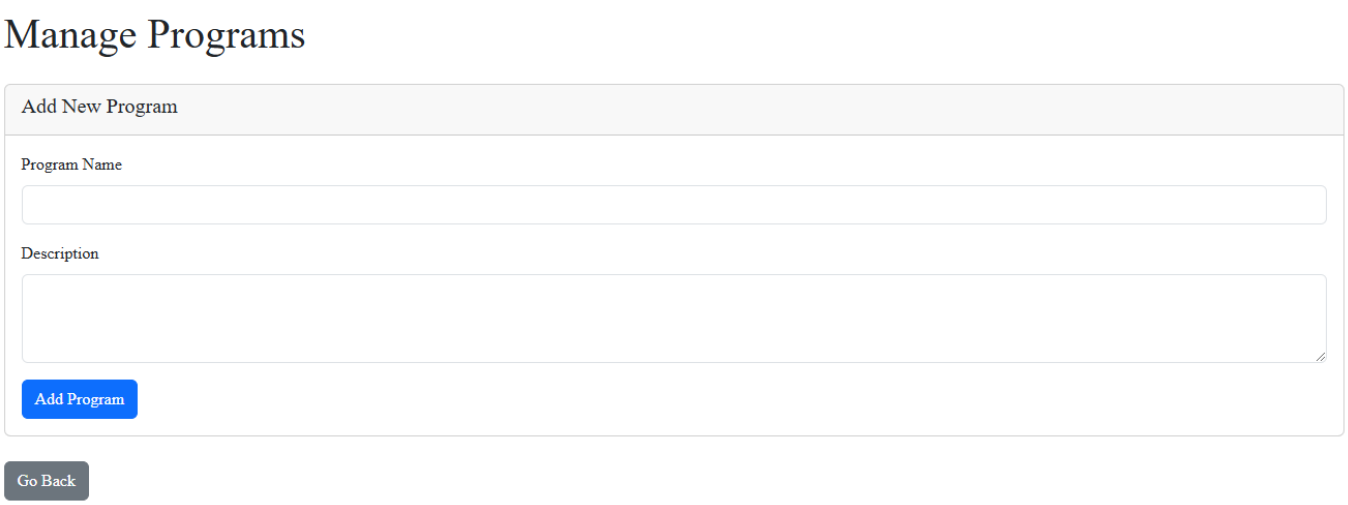
Πατήστε το κουμπί **Trainers** (http://localhost:3000/admin/manage-trainers) για να εμφανιστεί η σελίδα όπου μπορείτε να προσθέσετε τους γυμναστές με τις ειδικότητές τους (**Go Back** επιστροφή στο ταμπλό του **Manage System Structure**)



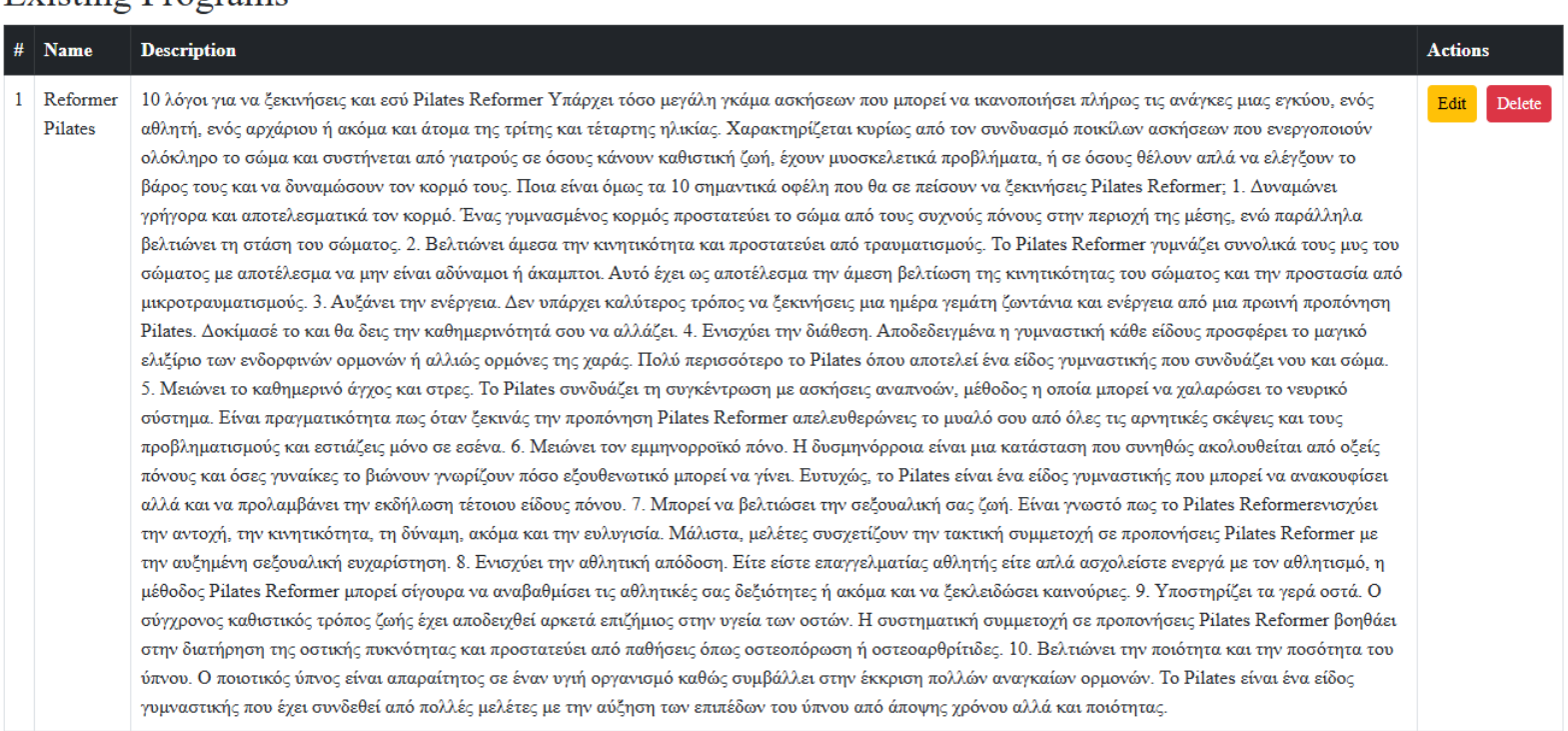
Μπορείτε επίσης, να δείτε τους ήδη υπάρχοντες και να επεξεργαστείτε τα στοιχεία τους ή να τους διαγράψετε.



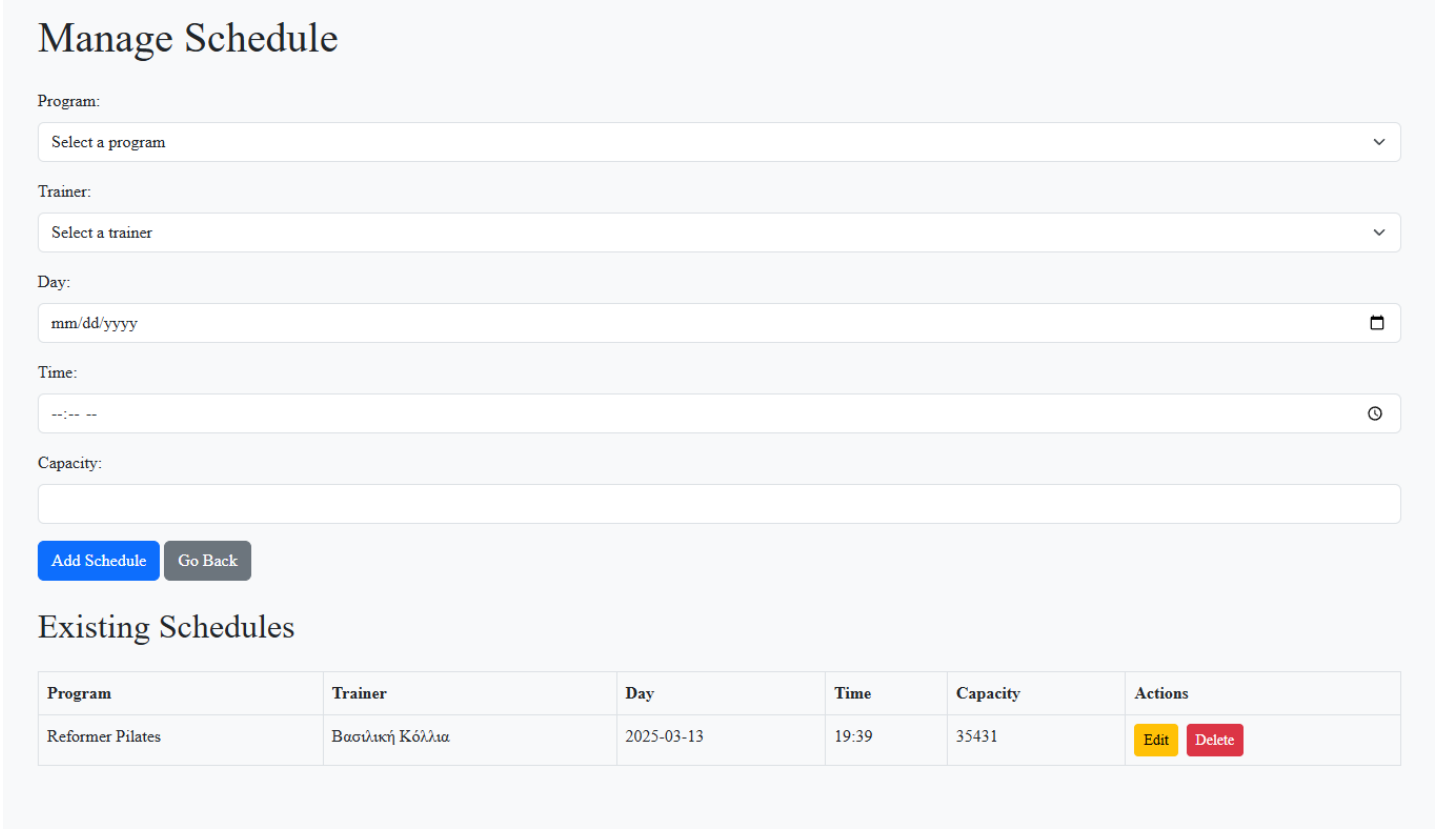
Πατήστε το κουμπί **Gym Programs** (http://localhost:3000/admin/manage-gym-programs) για να εμφανιστεί η σελίδα όπου μπορείτε να προσθέσετε προγράμματα γυμναστικής (**Go Back** επιστροφή στο ταμπλό του **Manage System Structure**).



Μπορείτε επίσης, να δείτε τα ήδη υπάρχοντα και να επεξεργαστείτε τα στοιχεία τους ή να τους διαγράψετε.



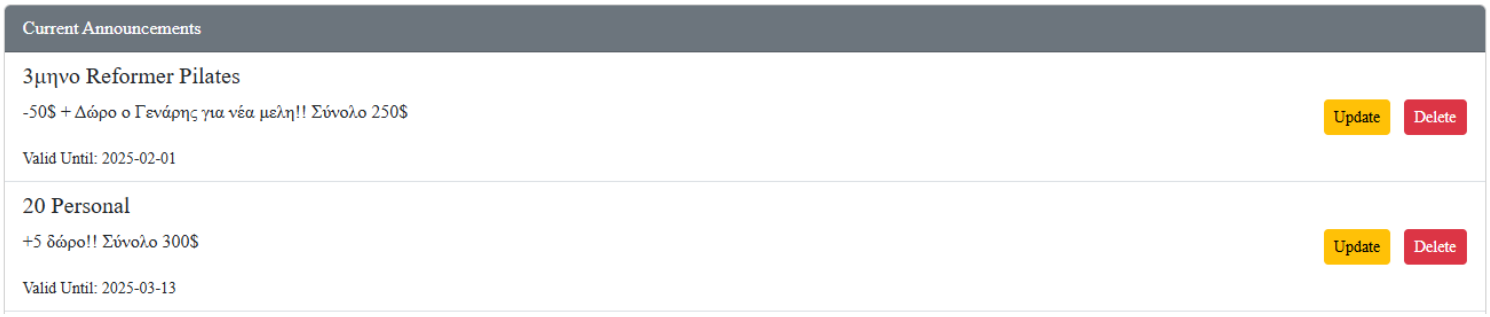
Πατήστε το κουμπί **Shedule** (http://localhost:3000/admin/manage-schedule) για να εμφανιστεί η σελίδα όπου μπορείτε να προσθέσετε το πλάνο των προγραμμάτων (ημέρα, ώρα, χωρητικότητα, γυμναστής)
(**Go Back** επιστροφή στο ταμπλό του **Manage System Structure**).
Μπορείτε επίσης να επεξεργαστείτε τα ήδη υπάρχοντα πλάνα ή και να τα διαγράψετε.



Πατώντας το κουμπί **Manage of Announcements/Offers** (απ' το http://localhost:3000/admin/dashboard) ανοίγει η σελίδα http://localhost:3000/admin/manage-announcements όπου μπορείτε να δημιουργήσετε μια ανακοίνωση ή να προσθέσετε μια προσφορά του γυμναστηρίου.



Επιπλέον μπορείτε να επεξεργαστείτε τις ήδη υπάρχουσες ανακοινώσεις/προσφορές.



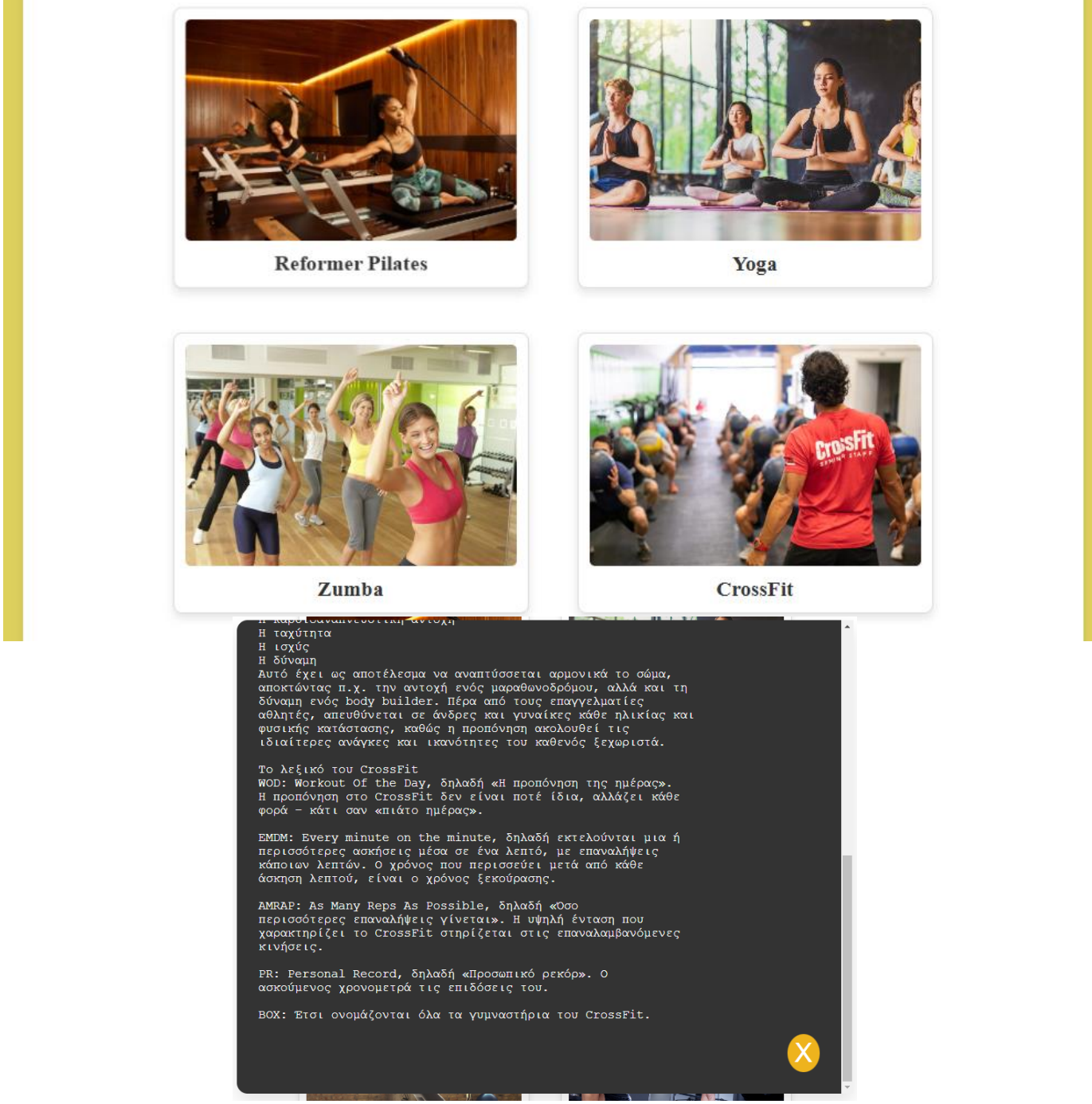
Με το κουμπί **Logout** αποσυνδέεστε απ' το σύστημα και επιστρέφετε στην αρχική σελίδα **http://localhost:3000.**

User

Αφού ο συνδεθείτε ως απλός χρήστης ανακατευθύνεστε στην διεύθυνση **http://localhost:3000/user/dashboard**

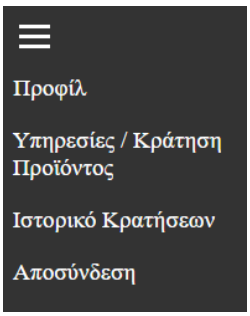


Σε αυτή την σελίδα ο απλός χρήστης μπορεί να δει όλες τις αναρτημένες και ενεργές ανακοινώσεις/προσφορές του γυμναστηρίου.

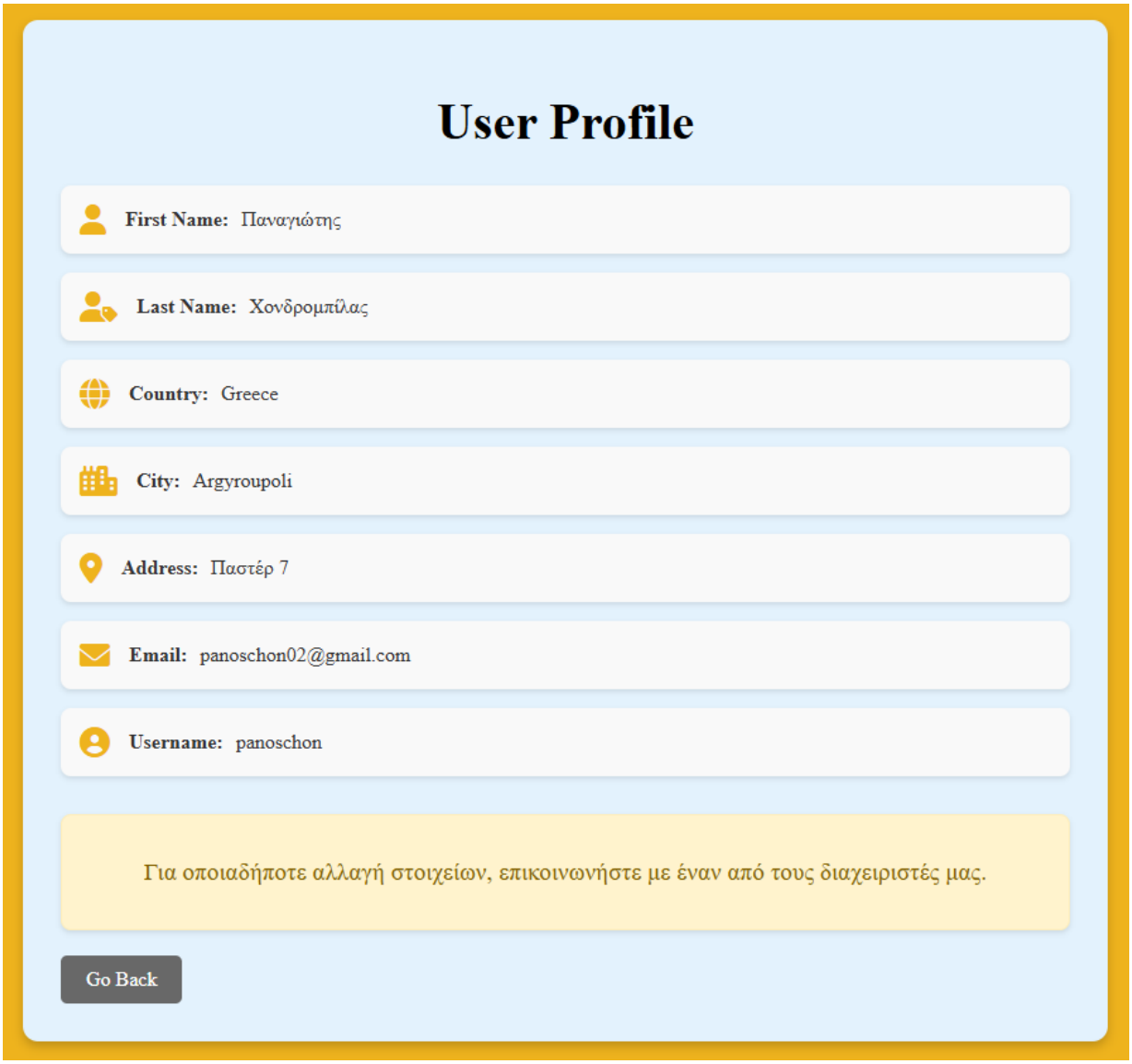


Επίσης μπορείτε να δείτε τις υπηρεσίες που προσφέρει, όπως και στην αρχική σελίδα **http://localhost:3000** με την διαφορά ότι τώρα, όταν πατήσετε πάνω σε μία εικόνα ανοίγει η περιγραφή του προγράμματος (στο τέλος της περιγραφής μπορείτε να κλείσετε το παράθυρο).

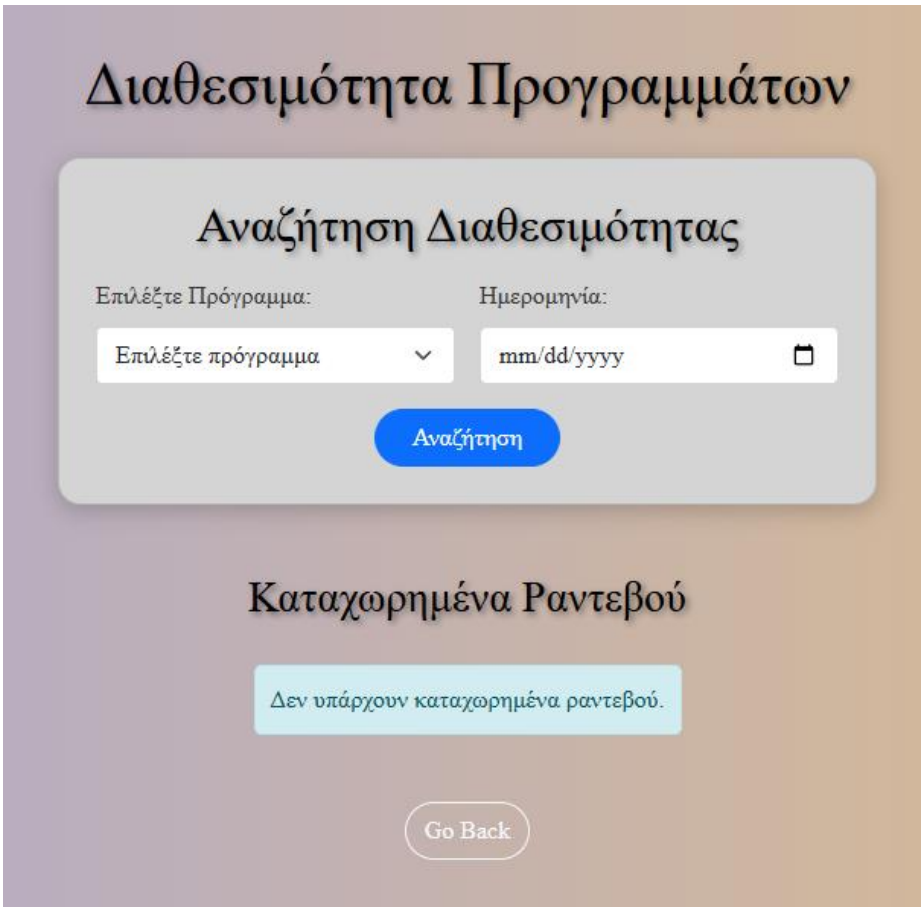
Πάνω αριστερά υπάρχει αυτό εδώ  ,το οποίο όταν το πατήσετε εμφανίζεται ένα ταμπλό στα αριστερά της οθόνης με τις ενέργειες που μπορεί να πραγματοποιήσει ο απλός χρήστης.



Επιλέγοντας το **Προφίλ** ανοίγει η σελίδα http://localhost:3000/user/profile με όλα τα στοιχέι του απλόυ χρήστη.



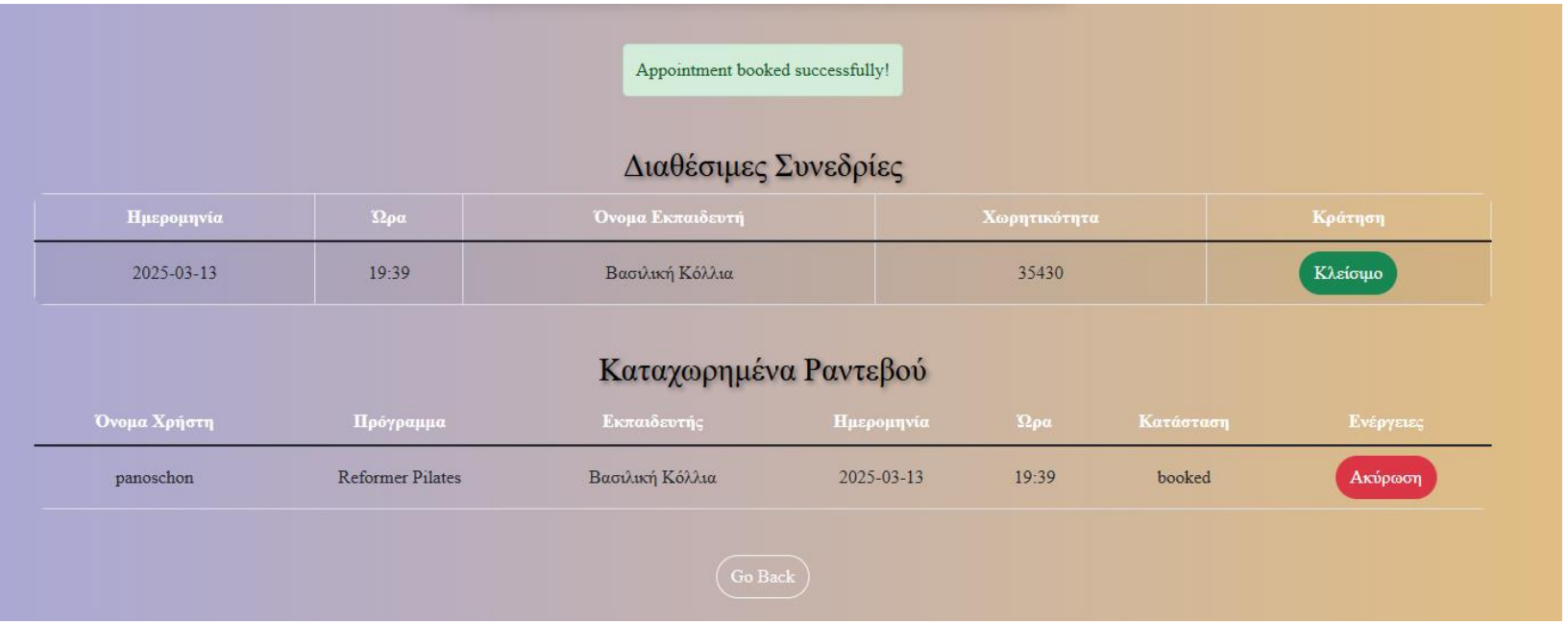
Επιλέγοντας τις **Υπηρεσίες/Κράτηση Προϊόντος** εμφανίζεται η σελίδα **http://localhost:3000/user/services**.



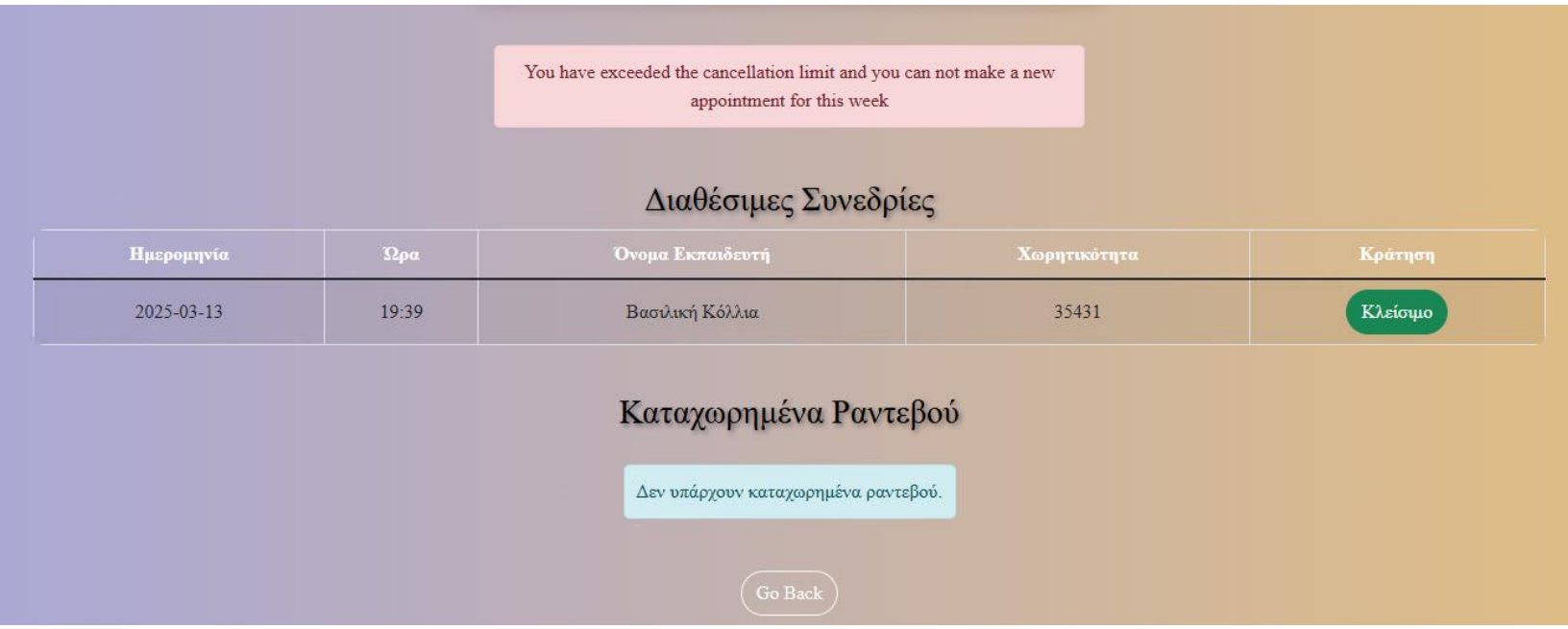
Εδώ επιλέγετε πρόγραμμα και ημερομηνία και πατάτε αναζήτηση.
Αν υπάρχει το πρόγραμμα στην επιλεγμένη ημερομηνία θα εμφανιστεί από κάτω.



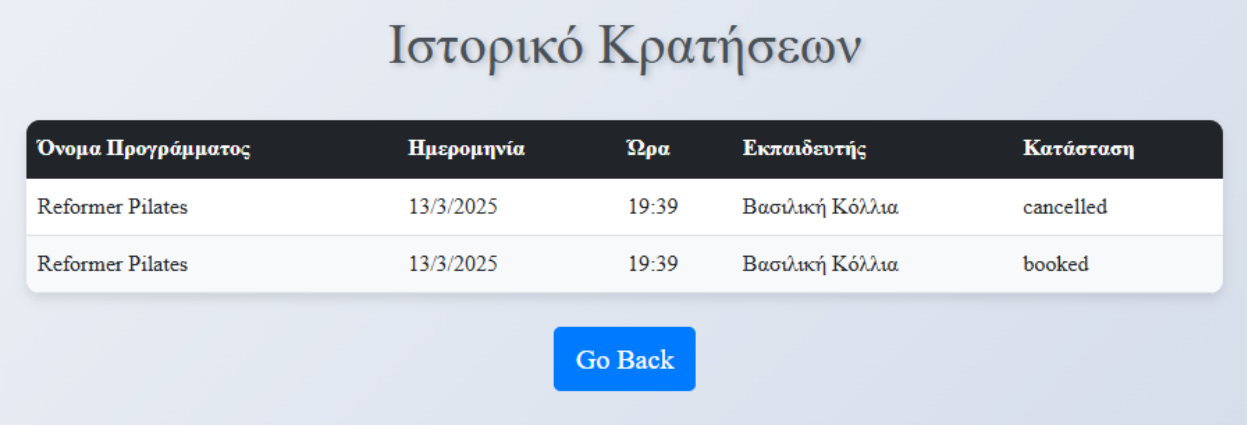
Πατώντας το **Κλείσιμο**, κλείνετε το ραντεβού σας για το συγκεκριμένο μάθημα, η χωρητικότητα μειώνεται κατά ένα και εμφανίζεται το ραντεβού στην λίστα **Καταχωρημένα Ραντεβού**. Αφού καταχωρηθεί το ραντεβού σας έχετε την δυνατότητα να το ακυρώσετε με το κουμπί **Ακύρωση.**



ΠΡΟΣΟΧΗ!!! Αν ακυρώσετε 2 ραντεβού μέσα στην ίδια εβδομάδα δεν θα μπορέσετε να κλείσετε άλλο ραντεβού για την υπόλοιπη εβδομάδα.



Επιλέγοντας το Ιστορικό Κρατήσεων εμφανίζεται η σελίδα **http://localhost:3000/user/booking-history**, όπου μπορείτε να δείτε όλα τα ραντεβού σας (ακυρωμένα ή ενεργά).



Με την επιλογή **Αποσύνδεση** αποσυνδέεστε απ' το σύστημα και επιστρέφετε στην αρχική σελίδα **http://localhost:3000**.

Στο σύστημα έχω δημιουργήσει τους εξής χρήστες:

Username	**Password**	**Role**
panoschon	321	admin
bilge	123	user
chalkiadakis	456	user
lena	789	user